

西安电子科技大学

硕士学位论文



嵌入式平台中的PCI Express总线技术研究

作者姓名 孙 铭

指导教师姓名、职称 武 斌 副教授

申请学位类别 工学硕士

学校代码 10701
分类号 TN919

学 号 1302120885
密 级 公开

西安电子科技大学

硕士学位论文

嵌入式平台中的PCI Express总线技术研究

作者姓名：孙 铭

一级学科：电子科学与技术

二级学科：电路与系统

学位类别：工学硕士

指导教师姓名、职称：武 斌 副教授

学 院：电子工程学院

提交日期：2015 年 12 月

Research on the Technology of PCI Express Used in Embedded Platforms

A thesis submitted to
XIDIAN UNIVERSITY
in partial fulfillment of the requirements
for the degree of Master
in Circuits and Systems

By
Sun Ming
Supervisor: Wu Bin Associate Professor
December 2015

摘要

随着信息技术的发展,数字信息资源的体积有了较大的膨胀,数字设备之间交换数据量越来越大。日益增长的数据量对高速数据传输接口的需求变得更加迫切。为满足这个需求,PCI Express 总线技术应运而生,并被广泛应用于个人电脑中。随着嵌入式系统性能的提升,PCI Express 总线技术在嵌入式领域也得到越来越广泛的应用。

嵌入式平台的主要核心基本上是处理器和可编程逻辑器件(FPGA)两部分,而且越来越多的处理器和FPGA支持高速PCI Express总线接口。基于此,本文研究了PCI Express技术在基于FPGA和处理器平台中的高速数据传输应用,设计并实现了一种FPGA与PowerPC处理器通信方案,并且在该方案的基础上,进一步对PCI Express技术在多个PowerPC处理器通信中的应用进行了探索和实践,最终实现了多个嵌入式PowerPC处理器的PCIe通信方案。在FPGA与处理器的PCIe通信系统中,由嵌入式PowerPC处理器作为根复合体(RC)设备,FPGA作为EP(端点)设备,二者直接建立PCIe链路进行通信;而在多个嵌入式PowerPC处理器的PCIe通信系统中,由一颗PowerPC处理器作为RC设备,其它PowerPC处理器作为EP设备,并通过交换芯片对PCIe链路进行扩展,建立起一对多的PCIe链路。所介绍的两种嵌入式平台的软件设计均基于嵌入式Linux系统,Linux应用程序通过PCIe设备驱动程序实现了对PCIe链路的初始化和控制。本文在两种嵌入式平台上所完成的工作有:

- 1.在FPGA与嵌入式PowerPC处理器的PCIe通信系统中,基于Xilinx公司的Virtex6系列FPGA的片内PCIe核实现了PCIe EP设备,与作为RC设备的MPC8377处理器建立PCIe链路。运行在MPC8377上的嵌入式Linux应用程序配合EP驱动程序访问FPGA,实现FPGA和PowerPC处理器的通信。

- 2.在多个嵌入式PowerPC处理器的PCIe通信系统中,实现了多颗MPC8377的PCIe通信。其中一颗处理器在switch上游作为RC设备,而其它处理器在switch下游作为EP设备。运行在RC端的嵌入式Linux应用程序配合EP驱动程序,实现了与EP处理器的通信。

论文分别给出了两种嵌入式平台运行测试的结果,证明其完全满足设计要求。

关键词: PCI Express 链路, 嵌入式系统, PowerPC 处理器, FPGA, Linux

ABSTRACT

With the development of information technology, the size of digital resources is increased, and the scale of data exchanging between digital devices is enlarging. The increasing size of data requires data transmission interfaces with higher bandwidth. To meet this requirement, PCI Express is put forward, and is used widely in PC. PCIe is still rarely used in embedded platforms, but it should appear in embedded fields more often with the enhancement of the performance of embedded systems. This paper researched on the application of PCIe in embedded platforms.

Firstly, the application of PCIe in FPGA and embedded PowerPC processor communication system is illustrated, and a kind of feasible plan is designed and implemented. Further more, based on this plan, the application of PCIe in multi-embedded PowerPC processors communication system is discussed, and a feasible plan is designed and implemented. Both of the two plans are put into practice. In FPGA and embedded PowerPC processor communication system, embedded PowerPC processor acts as RC device, while FPGA acts as EP device, a PCIe link is formed between them. And in multi-embedded PowerPC processors communication system, one PowerPC processor acts as RC device, and the others act as EP devices. The PCIe link is expanded by the switch chip, and a single to multiple PCIe link is formed. The software of the two systems are based on the embedded Linux operating system. The Linux applications initialize and control the PCIe links through the PCIe device drivers. The work on the two embedded platforms includes follows.

1. In FPGA and embedded PowerPC processor communication system, a PCIe EP device is implemented based on the PCIe IP core inside the Xilinx's Virtex6 family FPGA, and forms a PCIe link with the MPC8377 processor which acts as RC device. FPGA and PowerPC can communicate through the embedded Linux application and the EP device driver running on MPC8377.

2. In multi-embedded PowerPC processors communication system, the communication between multiple MPC8377s is implemented. One processor acts as RC in the upstream of the switch, while the others act as EPs in the downstream of the switch. The embedded

application and driver running on RC work together with the application running on EPs, and they can perform the communication between RC and EPs.

The running test results of these two kinds of embedded platforms are provided, which prove that they completely meet the designed requirements.

Keywords: PCI Express link, embedded system, PowerPC processor, FPGA, Linux

插图索引

图 2.1	PCIe 总线的层次划分	6
图 2.2	PCIe 的链路扩展	7
图 2.3	PCIe switch 的内部逻辑结构	7
图 2.4	89HPES24NT24G2 框图	8
图 2.5	89HPES24NT24G2 的多种工作模式	9
图 2.6	89HPES24NT24G2 的多分区工作模式	10
图 2.7	Virtex 6 PCIe 核功能框图	11
图 2.8	MPC8377 框图	12
图 2.9	MPC8377 的 PCIe 控制器框图	13
图 3.1	FPGA-PowerPC 通信方案结构框图	15
图 3.2	MPC8377 的地址/数据总线	17
图 3.3	MPC8377 的串行串解器物理层功能块框图	18
图 3.4	FPGA 逻辑功能框图	20
图 3.5	数据缓存模块框图	20
图 3.6	数据发送模块框图	21
图 3.7	RC 端应用程序流程图	40
图 3.8	系统启动信息	43
图 3.9	PCI 设备列表	44
图 3.10	装载设备驱动	44
图 3.11	PCIe 通信应用程序运行情况	45
图 3.12	PowerPC 接收到的递增码数据	46
图 4.1	多 PowerPC 通信方案结构框图	47
图 4.2	EEPROM 写事务时序图	52
图 4.3	89HPES24NT24G2 的工作模式	55
图 4.4	RC 端启动信息	65
图 4.5	EP 端启动信息	65
图 4.6	PCI 设备列表	66
图 4.7	装载设备驱动	66
图 4.8	设备文件列表	67
图 4.9	PCIe 通信应用程序运行情况	68

表格索引

表 2.1	各版本 PCIe 总线频率、峰值带宽和编码方式的差别.....	5
表 3.1	MPC8377 串行串解器的外部信号.....	19
表 3.2	串行串解器功能块控制寄存器组需要配置的寄存器字段.....	22
表 3.3	系统时钟配置寄存器需要配置的字段.....	24
表 3.4	PCIe 控制寄存器需要配置的字段.....	24
表 3.5	本地访问窗口基地址寄存器需要配置的字段.....	25
表 3.6	本地访问窗口属性寄存器需要配置的字段.....	25
表 3.7	CSB 桥控制寄存器需要配置的字段.....	26
表 3.8	Outbound 控制寄存器需要配置的字段.....	26
表 3.9	Inbound 控制寄存器需要配置的字段.....	26
表 3.10	Outbound 窗口属性寄存器需要配置的字段.....	27
表 3.11	Outbound 窗口基地址寄存器需要配置的字段.....	27
表 3.12	Outbound 窗口低地址译码寄存器需要配置的字段.....	27
表 3.13	Outbound 窗口高地址译码寄存器需要配置的字段.....	28
表 3.14	Inbound 窗口属性寄存器需要配置的字段.....	28
表 3.15	Inbound 窗口译码地址寄存器需要配置的字段.....	28
表 3.16	Inbound 窗口低基地址寄存器需要配置的字段.....	29
表 3.17	Inbound 窗口高基地址寄存器需要配置的字段.....	29
表 3.18	CSB 系统中断使能寄存器需要配置的字段.....	29
表 3.19	PCIe type1 配置头描述.....	30
表 3.20	DMA 写目的低地址寄存器.....	36
表 3.21	DMA 写目的高地址寄存器.....	36
表 3.22	DMA 控制寄存器需要操作的字段.....	38
表 3.23	DMA 中断状态寄存器关键字段.....	39
表 3.24	软复位寄存器需要操作的字段.....	41
表 3.25	中断屏蔽寄存器关键字段.....	41
表 3.26	DMA 写长度寄存器.....	42
表 3.27	功能控制寄存器关键字段.....	42
表 4.1	89HPES24NT24G2 的时钟配置管脚.....	49
表 4.2	89HPES24NT24G2 的 switch 模式配置管脚.....	49
表 4.3	单个双字初始化块的字节序列.....	50

表 4.4	配置完成块字节序列	51
表 4.5	EEPROM 写入事务字节序列.....	51
表 4.6	CCODE 字节定义	51
表 4.7	CMD 字节定义.....	52
表 4.8	Switch 控制寄存器需要配置的字段.....	53
表 4.9	分区 0 控制寄存器需要配置的字段	54
表 4.10	端口控制据存钱需要配置的字段	54
表 4.11	Inbound 窗口译码地址寄存器需要配置的字段	56
表 4.12	BAR 空间大小寄存器需要配置的字段	56
表 4.13	BAR 选择寄存器需要配置的字段	56
表 4.14	BAR 预取寄存器需要配置的字段	56
表 4.15	PCIe type0 配置头描述	57

符号对照表

符号	符号名称
MHz	兆赫兹
Gbps	吉比特每秒
MB	兆字节每秒
V	伏特
GT/s	千兆传输每秒

缩略语对照表

缩略语	英文全称	中文对照
AGP	Accelerated Graphic Ports	加速图形接口
BAR	Base Address Register	基址寄存器
DDR	Double Data Rate	双倍速率同步动态随机
DMA	Direct Memory Access	直接内存访问
EP	End point	端点
FIFO	First In First Out	先入先出队列
FPGA	Field Programmable Gate Array	现场可编程门阵列
PCI	Peripheral Component Interconnect	外围器件互联
PCIe	Peripheral Component Interconnect Express	高级外围器件互联
RAM	Random Access Memory	随机存取存储器
ROM	Read Only Memory	只读存储器
RC	Root Complex	根复合体
SDRAM	Synchronous Dynamic Random Access Memory	同步动态随机存储器

目录

摘要	I
ABSTRACT	III
插图索引	V
表格索引	VII
符号对照表	IX
缩略语对照表	XI
目录	XIII
第一章 绪论	1
1.1 课题研究背景	1
1.2 PCI Express 技术的发展现状	1
1.2.1 PCI 和 AGP 技术的发展	1
1.2.2 PCI Express 技术的发展现状	3
1.3 本文主要工作以及内容安排	4
第二章 PCI Express 及其链路扩展	5
2.1 PCI Express 技术简介	5
2.2 PCIe 链路扩展	6
2.3 FPGA 的 PCIe 接口设计	11
2.4 嵌入式处理器中的 PCIe 接口	12
2.5 本章小结	14
第三章 FPGA 与 PowerPC 处理器的 PCIe 通信设计	15
3.1 PCIe 通信方案总体设计	15
3.2 PCIe 通信方案硬件设计	16
3.2.1 FPGA 系统硬件设计	16
3.2.2 嵌入式 PowerPC 系统硬件设计	17
3.3 FPGA 的逻辑设计	20
3.4 PowerPC 软件设计	22
3.4.1 MPC8377 PCIe 控制器配置	22
3.4.2 EP 设备的驱动程序设计	30
3.4.3 RC 端应用程序设计	40
3.5 PCIe 通信功能测试	43
第四章 多个 PowerPC 处理器的 PCIe 通信设计	47

4.1	PCIe 通信方案总体设计	47
4.2	PCIe 通信方案硬件设计	48
4.2.1	PowerPC 部分硬件设计	48
4.2.2	PCIe switch 部分硬件设计	48
4.3	PCIe 通信方案软件设计	50
4.3.1	PCIe switch 配置	50
4.3.2	MPC8377 PCIe 控制器配置	55
4.3.3	EP 设备的驱动程序设计	58
4.3.4	RC 端应用程序设计	62
4.3.5	EP 端应用程序设计	64
4.4	PCIe 通信功能测试	64
第五章	总结与展望	69
参考文献		71
致谢		75
作者简介		77

第一章 绪论

1.1 课题研究背景

借力于信息技术的高速发展，信息化设备已被广泛应用于工业生产、安防监控、交通运输、影音娱乐、航空航天等诸多领域^[1]。互联网时代的到来也加速了数字信息资源的传播，数据存储设备的数量和规模也随之上升。越来越多的数据量对数据传输接口出新的要求。更高带宽，更大吞吐量的接口标准不断被推出。

数据传输接口带宽不足的问题首先出现在蓬勃发展的个人电脑领域。旧的低速数据传输接口已无法胜任海量数据的收发。对于新的数据传输接口，不但对其带宽有更高的需求，良好的通用性与兼容性也是必须具备的。此外数字信号的传输接口还必须具备较强的抗电磁干扰能力和容错率^[2]。为满足这些需求，PCI Express（简称 PCIe）总线标准应运而生。

PCI Express 在个人电脑中的应用取得了成功，而该技术在嵌入式平台中有着广阔的应用前景。嵌入式技术的迅速发展，使得多媒体设备极大的多元化，不同设备之间互传的数据量也变得非常可观，而 PCI Express 就是为了解决大量数据高速传输的问题而生的。研究 PCI Express 在嵌入式平台中的应用很有意义。随着嵌入式设备数量和规模的不断增长，一对一通信的形式已无法满足应用需求，一对多、多对多的通信形式已成为主流。而要实现多设备之间的 PCIe 通信，就要用到 PCIe 交换（switch）器件^[3]。目前对 PCIe switch 器件应用在嵌入式平台的研究还有很多空白。

1.2 PCI Express 技术的发展现状

1.2.1 PCI 和 AGP 技术的发展

PCI 即 Peripheral Component Interconnect(外围器件互联)^[4]，是一种同步复用的 32 位总线，蓬勃发展的个人计算机直接导致了它的诞生。有关 PCI 总线的工作始于上世纪 90 年代的 Intel 架构研发实验室。一些架构研发实验室的工程师（他们是该实验室的主力）定义了 PCI 总线架构，并与 Intel 的桌面计算机部门和核心逻辑部门的其他团队一起，开发出用于评估系统可行性的概念型芯片组和验证平台。

20 世纪 90 年代，PCI 总线被提出后，它迅速被应用于服务器当中以取代 MCA 总线和 EISA 总线，成为扩展服务器的可选总线之一。在主流个人电脑领域，PCI 总线淘汰 VESA 本地总线的步伐则要更慢一些，并且直到 1994 年第二代奔腾主机推出之前他都未能得到来自市场的足够的重视。到了 1996 年 VESA 本地总线被完全淘汰

时,个人电脑制造商们才纷纷在自己的产品上采用 PCI 总线,即便是 486 主机也不例外。EISA 总线则继续与 PCI 总线共存至 2000 年。苹果公司于 1995 年为其专业版 Power Macintosh 计算机引入了 PCI 总线。64 位版本的 PCI 总线在实际产品中很少应用^[5],但苹果为 Power Macintosh G3 和 G4 系列的所有产品都搭载了 64 位 PCI 总线。

1992 年 6 月 22 日, Intel 的架构研发实验室正式发布 PCI V1.0 规范。作为 Intel 的一个重要的实验室,该实验室曾先后提出过 AGP、USB、PC 的南北桥结构和 PCI Express 总线规范。

作为一种局部并行总线标准,PCI 总线成为了 ISA 总线的替代品,而前者是由后者发展出来的^[6]。随后的几个 PCI 版本又追加了一些新特性并在性能上有所提升。新的总线版本包括 66MHz 的标准 PCI 总线和 133MHz 的 PCI-X 总线,以及其他一些 PCI 总线信号形式的适配^[7]。PCI-X 规范是以 PCI 总线规范为基础进一步发展出来的。PCI-X 1.0b 和 PCI-X 2.0 都可以与标准 PCI 兼容。

PCI Express 总线在 2004 年问世的同时,PCI 总线被更名为传统 PCI(Conventional PCI)。在那之后,主板制造商逐渐减少了产品中的传统 PCI 插槽,取而代之的是新标准的接口。近几年推出的许多新主板甚至完全不提供传统 PCI 插槽。尽管如此,PCI 的辉煌还在延续,因为 PCI Express 继承了很多 PCI 设计思想,其很多特性也是由 PCI 演变而来。PCI/PCI Express 总线已成为目前绝大多数处理器系统用来连接通用外部设备的总线^[8]。

然而随着计算机越来越面向图形化,GPU 规格和运算能力不断提高,要提供能够满足显卡需求的带宽,PCI 总线已力不从心,这导致了专注于图形设备的 AGP 总线的发展。

AGP(Accelerated Graphic Ports,加速图形接口)是一种为显卡和计算机系统提供连接的高速点对点传输通道^[9]。它原本是被设计为专为显卡服务的 PCI 类接口的继任者的。到 2004 年,AGP 已基本被 PCI Express 总线取代。而到了 2008 年,PCI Express 显卡已经主宰市场,而 AGP 显卡已不多见。

AGP 规范主要的应用场合是 x86 个人电脑。与 PCI 总线相比,AGP 总线最大的有点在于它可以在插槽和处理器之间建立专有的链路,而不像 PCI 那是共享带宽的总线。此外,这种直连的连接方式允许总线时钟频率更高。AGP 还使用了边带寻址,这意味着地址总线 and 数据总线是分离的,这样不需要读取数据包就能得到地址信息。这个特性是通过额外添加的 8 位边带总线实现的,边带总线允许图像控制在发起 AGP 请求和命令的同时,其他 AGP 数据还能够在 32 位的主总线上传输。这导致了 AGP 数据吞吐能力的综合提升。此外,要加载一副纹理,PCI 显卡必须将纹理从系统 RAM 拷贝到显存,而 AGP 显卡可以通过图形地址映射表直接读取系统 RAM 中的纹理。图形地址映射表根据需要重新分配内存空间以存储纹理,这使得显卡能够直接访问纹

理。

AGP 插槽最早出现在基于 X86 兼容系统主板上, Intel 在 1997 年为 i440LX 芯片组引入了对 AGP 的支持, 并随在主要的主板厂商中之掀起了 AGP 浪潮。

1.2.2 PCI Express 技术的发展现状

计算系统 I/O 接口速率的不断提高催生了 PCI Express 总线技术。PCI 总线能够满足 USB 1.1、声卡以及 10/100M 网卡等外设的带宽需求。随着电子通信技术突飞猛进的发展, 千兆以太网、万兆以太网、USB3.0 等新一代的 I/O 技术标准如雨后春笋般的发展, PCI 总线的数据传输能力已跟不上节奏^[12]。PCI 总线渐渐成为了限制系统性能的瓶颈^[13]。为了满足现代计算机对总线带宽的需求, PCI Express 总线应运而生。PCI Express 总线采用高速串行传输技术, 该技术能够为总线提供极高的传输带宽^[14]。目前, PCI Express 总线技术在已经在个人电脑中得到广泛应用。

3D 显示卡技术的高速发展的使得显卡的规格越来越高, 显卡已经不得不抛弃 PCI 总线, 转而使用 PCIe 总线了。不仅是显卡, 声卡、千兆网卡和 RAID 卡也都逐渐发展出支持 PCI Express 接口的设备^[11]。老旧的 PCI 总线已经难以跟上 PC 技术发展的节奏, 其 533MB/s 的带宽已经难以承受高速以太网以图形加速卡等外设。而且, 外设规格的提升了带来了总线工作频率的提升, 总线功耗必然增大, 这就需要降低工作电压以解决功耗问题, 而 PCI 总线的工作电压难以降低。若在不进行更新换代, 陈旧的 PCI 总线必将制约 PC 系统的性能提升。

PCI Express 是为了取代老旧的 PCI、PCI-X 和 AGP 总线而设计出的高速串行计算机扩展总线标准^[10]。相对于较旧的总线标准, PCI Express 有了一系列的提升, 包括更高的最大系统总线吞吐量、更低的 I/O 管脚数目和更小的物理封装、总线设备的性能更高, 而且错误侦测与报告机制更加细化, 并增加了原生热插拔功能。一些近期的 PCI Express 版本还提供对 I/O 虚拟化的支持。PCI Express 的电气接口也被其它一些标准所使用, 如作为笔记本电脑扩展卡接口的 ExpressCard, 以及作为计算机存储接口的 SATA Express。PCI Express 由 PCI-SIG 奠基和开发的, 改组织的成员多达 900 多个公司, 这些公司也是传统 PCI 标准的奠基人。目前 PCI Express 3.0 标准的接口已应用于主流个人电脑中。2002 年 4 月 PCI Express v1.0 规范完成草案的制定, PCI Express v1.0a 规范与 2003 年正式发布。PCI Express2.0 规范于 2006 年正式发布。

目前, PCI Express v3.0 规范已经发布, 采用 128/130b 编码替代原先的 8/10b 编码, 相比 PCI Express v2.0 规范, 其总线带宽也提高了一倍, 再最高链路宽度的 x32 模式下, 其总线带宽高达 256Gbps。

1.3 本文主要工作以及内容安排

本文主要讨论了 PCI Express 技术两种嵌入式平台中的应用。在 FPGA 与嵌入式 PowerPC 处理器的 PCIe 通信系统中,嵌入式 PowerPC 处理器作为 RC 设备,而 FPGA 通过其内部 PCIe 核实现 EP 设备,并与 RC 建立 PCIe 链路进行通信。而在多个嵌入式 PowerPC 处理器的 PCIe 通信系统中,由一颗 PowerPC 处理器作为 RC,其他 PowerPC 处理器作为 EP,通过 switch 对 PCIe 链路进行扩展,实现一对多的 PCIe 通信。两种嵌入式平台的软件设计均基于运行在 PowerPC 上的嵌入式 Linux 操作系统。Linux 应用程序配合 PCIe 设备驱动程序,在软件层面实现 RC 设备对 EP 设备的访问。

论文的具体结构安排如下:

第一章介绍了论文研究的背景以及 PCI Express 技术的发展现状。

第二章介绍了 PCI Express 协议规范和技术指标,以及 PCI Express switch 技术的一些细节。

第三章研究了 FPGA 与嵌入式 PowerPC 处理器的 PCIe 通信系统的设计与实现,并讨论该 PCIe 通信系统的软硬件设计,其中软件设计部分是研究的重点。在第三章中,详细介绍了对 PowerPC 的 PCIe 控制器的配置、PCIe 设备驱动程序的设计和 PCIe 通信应用程序的设计,并对 PCIe 通信功能进行了测试。

第四章研究了多个嵌入式 PowerPC 处理器的 PCIe 通信系统的设计与实现,并详细描述了用 switch 对 PCIe 链路进行扩展的具体实现方法,以及 RC 端与 EP 端 PowerPC 的配置、驱动程序设计和应用程序设计,并对 PCIe 通信功能进行了测试,验证了该方案的可行性。

第五章回顾本文内容,总结了本文所完成的工作,并分析了本文的研究成果,对一些不足之处提出了改进方向。

第二章 PCI Express 总线及其链路扩展

2.1 PCI Express 技术简介

随着对总线带宽要求的不断提高，并行总线在工作频率上的瓶颈日益凸显，高速串行总线逐渐取代了并行总线。作为 PCI 总线的替代品，PCIe 的串行差分信号线工作频率更高，使用的信号线更少，总线带宽也更高。

PCIe 采用 end-to-end 的连接方式，即一条链路只能提供一对设备的连接。每条物理链路可由若干条数据通路（lane）组成。目前一条 PCIe 链路可支持 1、2、4、8、16、32 个 lane（即 x1，x2，x4，x8，x16，x32 宽度的 PCIe 链路）^[15]。

根据 PCIe 总线规范的不同版本，lane 有多种总线频率，链路编码方式也有所差别^[16]，具体情况如表 2.1 所示。

表2.1 各版本 PCIe 总线频率、峰值带宽和编码方式的差别

PCIe 版本	总线频率	Lane 峰值带宽	链路编码方式
1.x	1.25GHz	2.5GT/s	8/10b 编码
2.x	2.5GHz	5GT/s	8/10b 编码
3.0	4GHz	8GT/s	128/130b 编码+

按照表 2.1 中的数据计算，PCIe 3.0 x32 链路可提供高达 256GT/s 的峰值带宽。尽管 PCIe 的链路带宽并不能 100%的转化为有效带宽，但是其有效带宽在目前看来还是相当高的。

PCIe 总线虽在带宽上远超目前的绝大多数总线，但也存在传输延迟较大的缺点。PCIe 信号在传输线上是串行的，但在 PCIe 控制器中，还是需要串行串解器（SerDes）对信号进行串并转换，因为处理器、PCIe 控制器中的寄存器、总线和内存空间当中的数据都是并行形式的。而串并转换的过程会产生较大延时。

PCIe 体系结构中的主要部件包括 RC（Root Complex，根复合体）、EP（Endpoint，端点）、switch 等。在 PCIe 总线中，RC 相当于包含 PCIe 总线控制器的主机，而 EP 是基于 PCIe 总线的设备，switch 则类似于以太网中的交换机，可实现 PCIe 的链路扩展。

PCIe 总线进行数据传输时的最小逻辑单元是数据包（packet）。与 OSI 网络的 7 个层次类似，PCIe 总线也分为多个层次。数据报文发送和接收的过程中，要依次经过核心层（Device Core）、事务层（Transaction Layer）、数据链路层（Data Link Layer）

和物理层（Physical Layer）^[17]。数据报文由核心层生成，最终由物理层发出，接收报文的过程则相反。

事务层定义了 PCIe 总线使用的总线事务。PCIe RC 设备和 EP 设备可以用这些总线事务互相访问。总线事务也可以被 switch 转发。在事务层主要的工作是 TLP（Transaction Layer Packet）的封装和解包。对于发送操作，事务层将核心层的数据封装成 TLP 并发送至数据链路层；对于接收操作，事务层接收从数据链路层传递的 TLP，解包后发给核心层。

数据链路层为来自事务层的报文添加序号前缀和 CRC 后缀^[18]。为保证报文传输的可靠性，数据链路层采用了 ACK/NAK 协议。

在 PCIe 总线的物理层中，实现了总线的物理连接。物理层还实现了 LTSSM（Link Training and Status State Machine）状态机，进行电源管理、链路训练和链路恢复^[19]。物理层可细分为逻辑子层和电气子层。PCIe 总线的层次划分如图 2.1 所示。

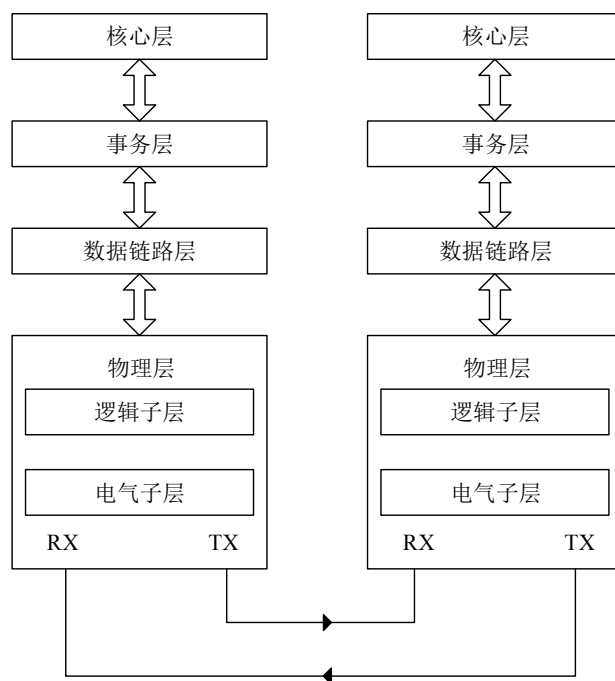


图2.1 PCIe 总线的层次划分

2.2 PCIe 链路扩展

PCIe 使用的数据连接方式是 end-to-end。在一条 PCIe 链路中，连接发送和接收设备的两个端口是完全对等的。与以太网链路可以使用交换机扩展类似，PCIe 链路也可以用 switch 扩展，以连接多个设备。PCIe 链路扩展的一般形式如图 2.2 所示。

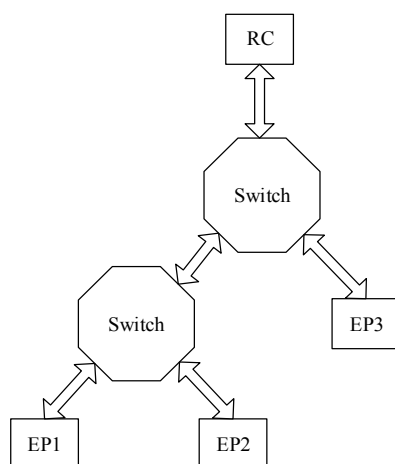


图2.2 PCIe 的链路扩展

在 PCIe 总线中，能够直接或间接与 RC 相连的端口称为上游（upstream）端口，其余的端口都称为下游（downstream）端口。除了可以扩展 RC 与 EP 的连接，switch 还提供对 RC 与 RC 连接的支持。实现这种连接方式有两种方法：Crosslink 和非透明桥。

一个标准的 switch 设备具有一个上游端口和多个下游端口。上游端口与 RC 或其他 switch 的下游端口连接，下游端口与 EP 或其他 switch 的上游端口连接。在以 Crosslink 方式连接时，也支持上游端口与其他 switch 的上游端口连接，下游端口与其他 switch 的下游端口连接。在 switch 中还有 Egress 端口和 Ingress 端口的概念。Egress 端口指数据离开 switch 所用的端口，Ingress 指数据进入 switch 所用的端口^[15]。

switch 的内部结构，从软件层面来看，是由多个 P2P 桥（PCI-to-PCI bridge）组成的。switch 中的每一个上游端口和下游端口都对应一个虚拟 PCI 桥，每个 PCI 桥都有自己的配置空间。这些虚拟 PCI 桥由 switch 内部的一条虚拟 PCI 总线连接。PCIe switch 的内部逻辑结构如图 2.3 所示。

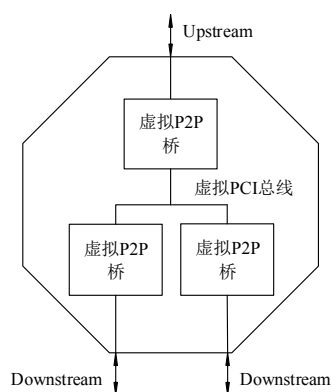


图2.3 PCIe switch 的内部逻辑结构

本文使用 89HPES24NT24G2 switch 芯片实现 PCIe 链路的扩展。该芯片是 IDT 公司推出的一款兼容 PCI Express 2.1 规范标准的 PCIe switch 桥片。支持 24 lane、24 端口的 PCIe switch 互联，每个 lane 的峰值带宽可支持到 5GT/s^[21]。

89HPES24NT24G2 支持非透明桥连接方式。非透明桥用于连接两个不同的 PCIe 总线域的场所，可以实现 RC 与 RC 的连接。

89HPES24NT24G2 有编号从 0 到 23 共 24 个端口，每个端口都支持在 Gen1（2.5GT/s）和 Gen2（5GT/s）模式下工作。switch 内部由 4 个 PCIe 栈（stack）、两个 DMA 模块、一个 switch 内核和 SMBus、GPIO、复位等外设块组成。一个 PCIe 栈包括符合 PCIe2.1 规范的事务层、数据链路层和物理层逻辑。此外，PCIe 栈用路由表实现了 TLP 路由，并能够处理读写配置请求，还能完成非透明桥地址译码的功能。这些功能构成了 switch 的应用层。89HPES24NT24G2 的框图如图 2.4。

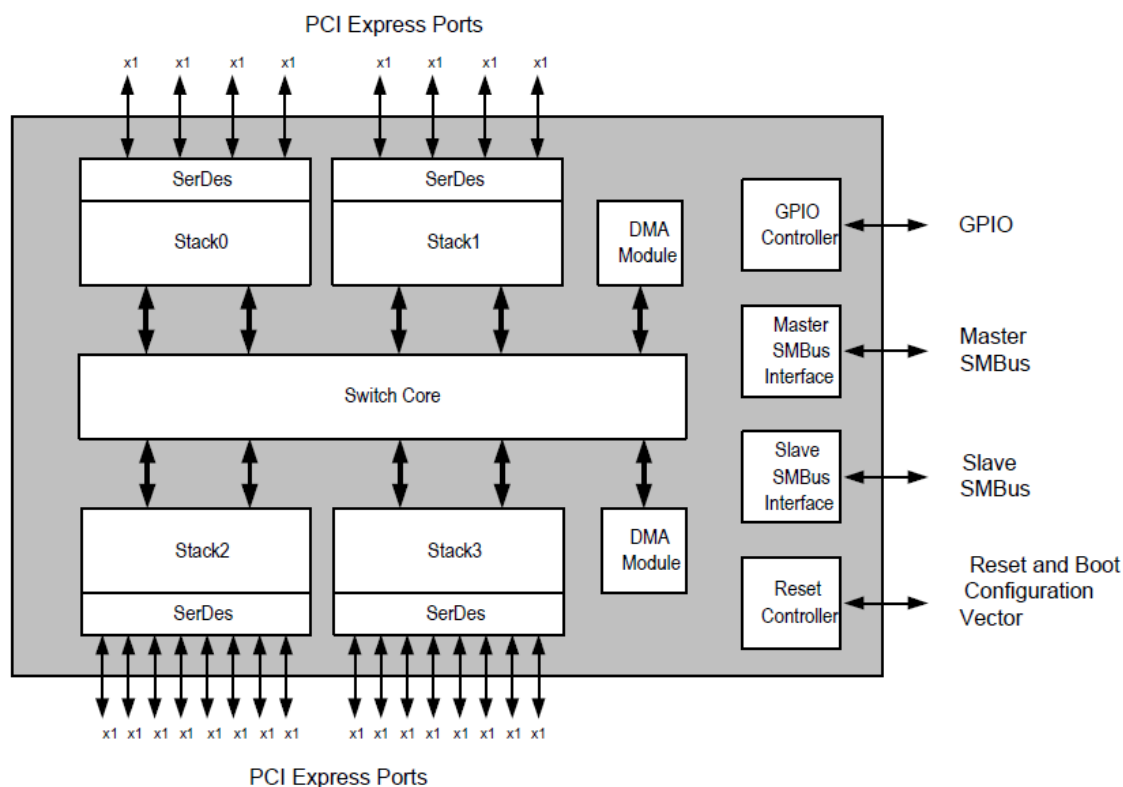


图2.4 89HPES24NT24G2 框图

在 89HPES24NT24G2 的 4 个 PCIe 栈中，stack1 和 stack2 各包含 4 个端口，这 4 个端口只能以 4 个独立的 x1 宽度端口的形式工作，而不能与相邻的端口合并成更高宽度的端口。Stack3 和 stack4 各包含 8 个端口，这 8 个端口可以合并为 1 个 x8 宽度的端口、2 个 x4 宽度的端口、4 个 x2 宽度的端口，或以 8 个 x1 宽度的端口的模式独

立工作，同时还支持多种宽度端口混合工作，如 8 个端口合并成 1 个 x4 宽度的端口和 2 个 x2 宽度的端口。

switch 内核负责在 4 个 PCIe 栈之间转发 TLP 包。它的主要功能是缓存输入的 TLP、保存 ingress 和 egress 的流控制信息和端口仲裁等。由于 switch 是为系统互联和扇出而优化的独立架构，所以 switch 内核被设计成基于非阻塞交叉开关矩阵（non-blocking crossbar）。

即使在同一个 PCIe 栈当中，栈中的各个端口可相互独立的工作。每个端口都有若干种工作模式。在不同的工作模式下，端口所属的 P2P 桥、非透明 EP、DMA EP 的功能也各不相同。89HPES24NT24G2 的端口支持如下工作模式：

Disabled（不使能）

Unattached（未连接）

Upstream switch port（上游 switch 端口）

Downstream switch port（下游 switch 端口）

Upstream switch port with DMA function（带 DMA 功能的上游 switch 端口）

Upstream switch port with NT function（带非透明传输功能的上游 switch 端口）

Upstream switch port with NT and DMA functions（带非透明传输功能和 DMA 功能的上游 switch 端口）

NT function（非透明传输）

NT with DMA function（带 DMA 的非透明传输）

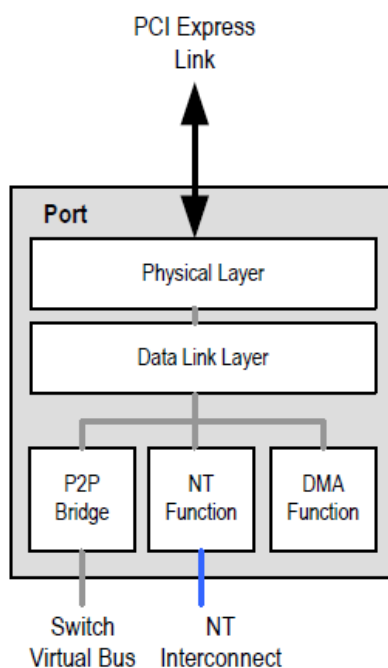


图2.5 89HPES24NT24G2 的多种工作模式

图 2.5 表明, 根据端口工作模式的不同, 一个端口可能包含一种、两种或最多三种功能 (这三种功能是 P2P 桥、非透明传输和 DMA)。有多个功能的端口只能是上游端口。端口具有多个功能时, 数据可以在这些功能模块之间传输。例如, 在 NT with DMA function 工作模式的端口中, TLP 包可以从 DMA 模块传递至非透明桥模块。跨功能模块传输只在端口内部进行, 并不表现在 PCIe 连接上。端口的链路宽度不是由端口的工作模式决定的, 而是由 PCIe 栈的配置决定。实际的工作链路宽度则是在链路训练时才最终确定的。

并非所有端口都支持所有的工作模式: 所有端口都支持 disabled、unattached、upstream switch port 和 downstream switch port 模式; 端口 0、2、4、6、8、12、16 和 20 支持带 NT function 的模式; 端口 0 和 8 支持带 DMA function 的模式。端口的工作模式可自由配置, 工作模式配置可以在器件启动时或运行时改变。

89HPES24NT24G2 是可分区的 PCIe switch。这意味着, 89HPES24NT24G2 不仅可以作为标准 PCIe switch 工作, 它的端口还可以分区独立工作, 每个区在逻辑上相当于一个独立的 PCIe switch。多分区工作方式如 2.6 所示。

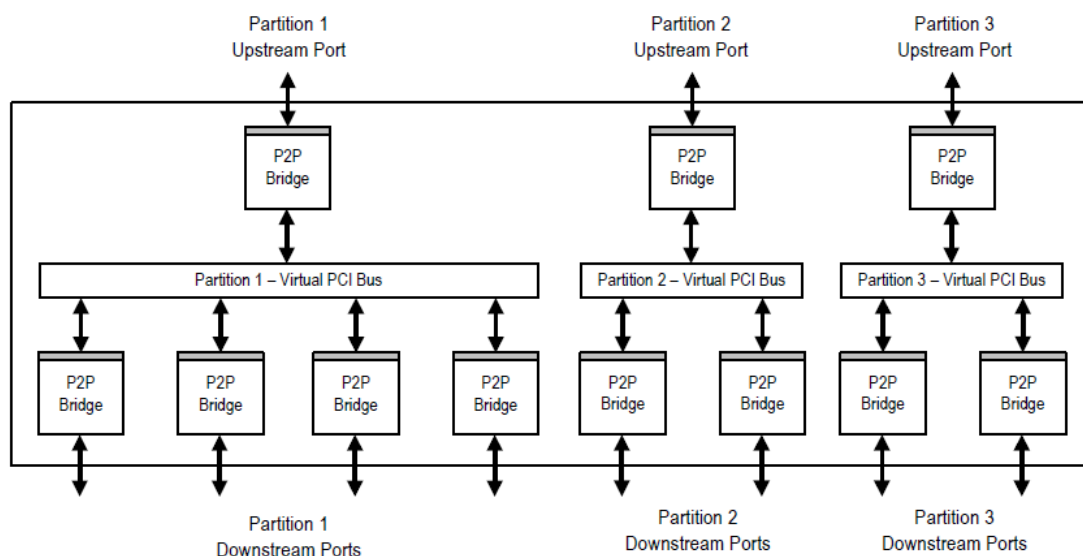


图2.6 89HPES24NT24G2 的多分区工作模式

每个 switch 分区的端口数量都可配置, 每个分区都连接到一个独立的 PCIe 域上。各个分区在逻辑上是孤立的。从 switch 的角度看, 一个 switch 分区表示一个逻辑容器, 它包含了与同一个 PCIe 域相关的一组端口的。89HPES24NT24G2 支持启动时配置端口分区和上电时配置端口分区。启动时配置可由 EEPROM 或外部 SMBus 主机完成。分区配置也可以在初始复位时通过 switch mode 引脚 (SWMODE[3:0]) 完成。运

运行时复位运行系统在已经运行的情况下改变分区数量和每个分区的端口。运行时配置可以由 SMBus 主机完成。

2.3 FPGA 的 PCIe 接口设计

FPGA 器件具有设计灵活，集成度高等特点，被广泛应用与嵌入式系统当中。随着 PCIe 总线的普及，越来越多的 FPGA 器件提供了对 PCIe 接口 IP 核的支持。基于 FPGA 的 PCIe 接口具有设计灵活、定制化程度高等优势，但与基于嵌入式处理器内嵌的 PCIe 接口相比，其设计难度相对较大。

本文基于 Xilinx 公司的 Virtex 6 系列 FPGA 进行 PCIe 接口设计。Virtex 6 系列 FPGA 内部集成 PCI Express 核，该核是具有性能靠性、带宽高、伸缩性强等特点的串行互联定制块，同时支持 VHDL 和 Verilog-HDL 语言开发。它在 Virtex 6 系列 FPGA 上实现了 PCI Express 总线功能。

Virtex 6 系列 PCIe 核完全支持工作在 2.5Gbps 和 5Gbps 下的 PCIe EP 配置和 RC 配置，最高支持 x8 链路宽度，符合 PCI Express 2.0 规范，并向下兼容。PCIe 核实现了系统接口、PCI Express 接口、配置接口、事务接口和物理层控制与状态接口，其结构框图如图 2.7 所示：

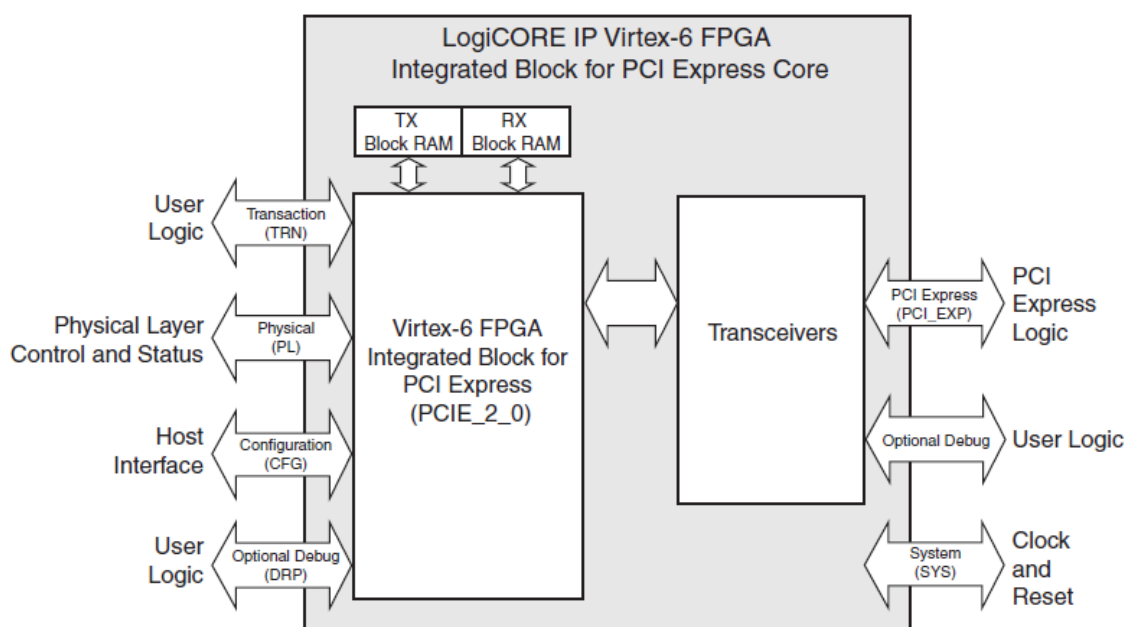


图2.7 Virtex 6 PCIe 核功能框图

PCIe 核使用数据包在各种模块之间进行信息交换。数据包的组装在事务层和数据链路层中完成。

用户通过 IP 核生成工具定制并生成 PCIe 核后，就得到了用户逻辑和 PCIe 总线逻辑的接口，要实现基于 PCIe 总线的通信功能，还需要用户自行设计逻辑功能块，与 PCIe 核通信，通过 PCIe 的各种用户接口控制 PCIe 核完成通信。这虽然提高了 PCIe 逻辑的设计难度，但体现了基于 PCIe 核进行设计的灵活性。

2.4 嵌入式处理器中的 PCIe 接口

为满足嵌入式系统对数据传输接口带宽的要求，在 ARM、PowerPC、MIPS 等架构的嵌入式处理器中，都出现了集成 PCIe 控制器的产品系列。这类处理器产品简化了嵌入式 PCIe 接口设计，使得 PCIe 接口在嵌入式领域中的得到了更加广泛的应用。

本文选用 MPC8377 处理器作为嵌入式通信系统的主控制器。MPC8377 是 Freescale 公司推出的 e300 内核 PowerPC 架构处理器，最高工作频率可达 800MHz，能够为互联型设备提供高性价比的解决方案。片上集成 32 KB 指令缓存和 32 KB 数据缓存，实现用户指令集架构并提供对软件和硬件的调试支持。外设接口包括两个三速以太网接口、一个 DDR1/DDR2 SDRAM 控制器、一条增强型本地总线（eLBC）、四个 SATA 控制器、一个 32 位 PCI 总线控制器、一个 PCI Express 总线控制器、USB2.0 主从控制器以及 I2C、GPIO 等^[20]。图 2.8 展示了 MPC8377 框图。

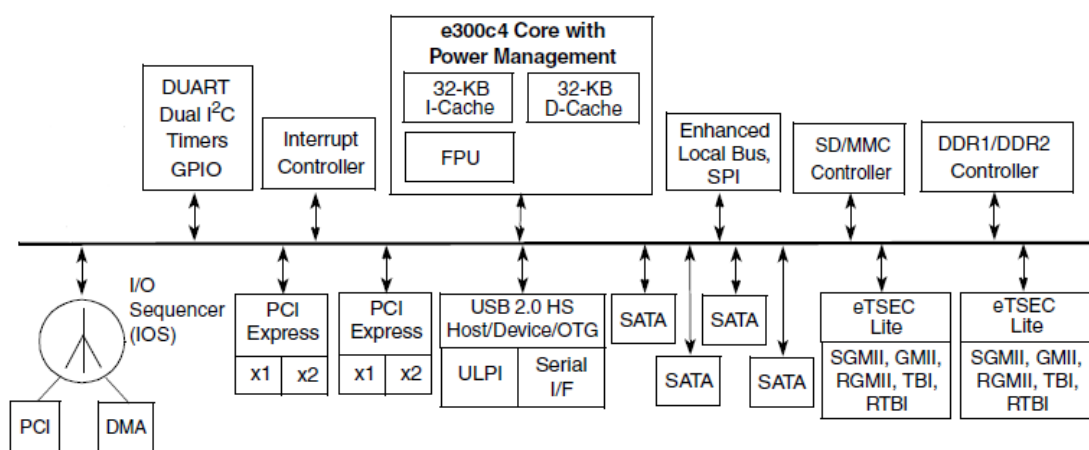


图2.8 MPC8377 框图

MPC8377 通过 PCIe 控制器实现一致性系统总线 CSB 和 PCIe 总线的接口。MPC8377 的 PCIe 接口兼容 PCI Express v1.0a 规范，其片内 PCIe 控制器相当于一个用于与 PCIe 设备通信的状态机。该 PCIe 控制器主要由三部分组成：

1、PCI Express 核，其功能是处理事务层、数据链路层、介质访问控制层的数据传输。同时 PCI Express 核也涵盖了 PCIe 配置头信息和 PCIe 控制寄存器。

2、CSB 桥，控制 CSB 总线与 PCIe 事物层之间的事务传输。除此之外，CSB 桥还包括一组 DMA 读写引擎、一个消息管理器和一组配置寄存器。

3、串行串解器，控制 PCIe 介质访问控制层与物理层之间的数据传输。串行串解器也有自己的一组配置寄存器。MPC8377 的 PCIe 控制器框图如图 2.9 所示。

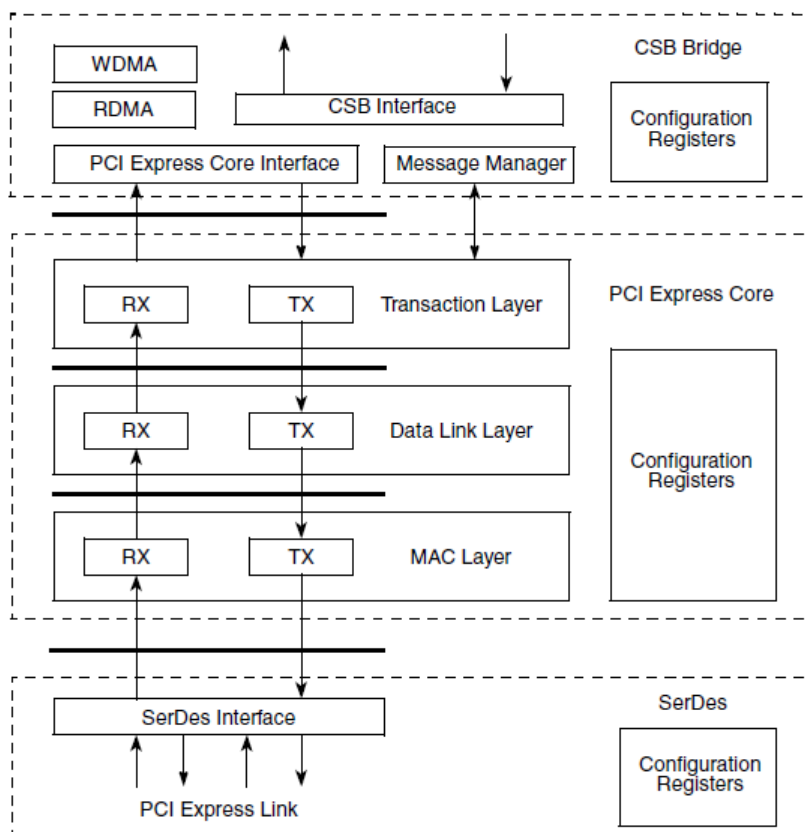


图2.9 MPC8377 的 PCIe 控制器框图

MPC8377 的 PCIe 控制器实现了 CSB 总线和 PCIe 总线的连接，其中 PCIe 总线最多支持 x2 lane 的 2.5GHz 串行通信。该控制器内部包含跟踪 inbound 和 outbound 事物的队列。其控制逻辑可处理缓冲区管理、总线协议、事务生成和标记产生，同时还具备额外的存储块，能够保存 inbound 和 outbound 数据。

MPC8377 可以被设置为根复合体（RC）模式或端点（EP）模式。RC 设备可将 CPU 或存储子系统与 I/O 设备连接，而 EP 设备通常表示一个外设或 I/O 设备。在 RC 模式下，使用 type 1 型配置头，而在 EP 模式下使用 type 0 型配置头。

作为总线通信的发起者，MPC8377 支持最大载荷（payload）128 字节的读写操作和 I/O 事务。此外，在 RC 模式下还支持 outbound 配置。作为目标接口设备，MPC8377 支持对本地内存空间的读写操作。而在 EP 模式下，MPC8377 可以接受对内部 PCIe 配置寄存器的配置事务，但不支持 inbound I/O 事务。

PCIe 总线的 outbound 事务首先被映射到地址译码窗口，以确定哪些事务是需要处理的。从 CSB 总线发出的事务可以是内存事务、I/O 事务或配置事务，这取决于译码窗口的属性。根据原始请求大小、事务类型、PCIe 设备控制寄存器的 Max_Payload_Size 域或 Max_Read_Request_Size 域，一个事务可被分为更小的事务。PCIe 控制器通过执行序规则检测来确定需要向 PCIe 总线发送的下一个事务。一般来说，事务是按照它们从 CSB 被接收到的顺序发送的。当有搁置条件时，PCIe 控制器允许优先级高的事务重新排列在优先级低的事务之前。对于 posted 写事务，在所有 CSB 总线上的数据都被接收之后，数据才被向前转发至 PCIe 总线，并认为该事务已完成。对于 non-posted 写事务，PCIe 控制器在收到对方的完成应答后才认为事务已完成。而对于 non-posted 读事务，PCIe 控制器要等待收到对方的所有完成应答后，再将所有数据发送至 CSB 总线并结束事务。

产生 PCIe outbound 事务的方式有两种：

- 1、某个 CSB 主机，如 e300 主控制器，直接发起一个事务。这被称为“PIO”。
 - 2、作为 PCIe 控制器 CSB 桥的一部分，读 DMA 引擎或写 DMA 引擎被使用。
- 在传输大块数据时，DMA 方式更加高效。Outbound 窗口可被这两种方式共享。

MPC8377 作为 PCIe 总线通信目的地时，向 CSB 发送的 PCIe inbound 事务首先通过地址译码窗口映射到 CSB 地址空间。在事务向 CSB 发送时，根据其原始大小、字节使能和起始地址，一个事务可被分为更小的事务。PCIe 控制器通过执行序规则检测来确定需要向 CSB 总线发送的下一个事务。一般来说，事务是按照它们从 PCIe 总线被接收到的顺序发送的。当有搁置条件时，PCIe 控制器允许优先级高的事务重新排列在优先级低的事务之前。对于 posted 写事务，在 PCIe 总线上的所有数据都被接收后，数据才被向前转发至 CSB 总线并认为该事务已完成。对于 non-posted 读事务，PCIe 控制器需等待足够的完成应答包返回（取决于包长度）在将数据转发至 PCIe 总线。这一过程将一直持续至没有需要发送的完成应答包为止。

MPC8377 的 PCIe 控制器支持 32 位寻址和 64 位寻址，可支持两个 x1 lane 的 PCIe 链路或一个 x2 lane 的 PCIe 链路。内存读写的最大载荷大小（Max_Payload_Size）128 字节。每路 PCIe 链路提供 4 个 inbound 传输译码窗口、4 个 outbound 传输译码窗口和一组读写 DMA 引擎。支持最大载荷 32 字节的来自 CSB 的事务。

2.5 本章小结

本章介绍了 PCI Express 总线的组成和特性，以及 PCIe switch 的一些细节。并结合 PCIe 总线规范介绍了具体器件上的 PCIe 部件。

第三章 FPGA 与 PowerPC 处理器的 PCIe 通信设计

随着信息时代的到来,在各类存储介质之间传输的数据量越来越大,海量的数据对数据传输接口的传输带宽不断提出新的挑战^[22]。使用传输速度更快、带宽更高的接口实现不同设备之间的通信也成为研究的热点。

传统的并行传输接口,虽然数据位宽较高,但是多条数据线之间容易发生串扰,并且数据线是单端形式,也容易收到外界干扰。为了提升并行接口的带宽,接口的工作频率也必须提高。而在高频率下,数据串扰等信号完整性问题尤为突出,严重限制了并行数据接口的工作频率上限。采用串行数据接口,可使数据线的数量大大减少,从而避开并行数据接口的信号完整性问题。高速串行接口采用的差分信号线,也可以削弱外界电磁环境对信号线的干扰,抗干扰能力相比并行数据接口有了很大提升。线路精简、性能强大的 PCI Express 总线非常适合解决嵌入式系统中的数据传输问题。

FPGA 的 I/O 资源丰富,配置灵活,可根据用户需要实现特定的逻辑功能,非常适合应用于嵌入式数据传输系统中。

本章实现了 FPGA 与嵌入式处 PowerPC 理器的 PCIe 通信。在该系统中, FPGA 通过 PCIe 核和用户 DMA 逻辑实现了 PCIe EP 设备,与作为 RC 设备的 MPC8377 PowerPC 处理器建立 PCIe 连接并通信。在本章结尾对该设计方案进行了测试,验证了该方案的可行性。

目前该方案已应用于星载高速串行数据专用检测设备中。

3.1 PCIe 通信方案总体设计

在该 PCIe 通信方案中,上游数据源由 FPGA 内部的数据生成模块产生。FPGA 先将数据缓存在外部 DDR3 RAM 中,再通过 PCIe 总线发送给 PowerPC, PowerPC 可挂载硬盘,并将数据存储存储在硬盘中,一般分析数据的正确性。通信系统方案结构框图如图 3.1 所示。

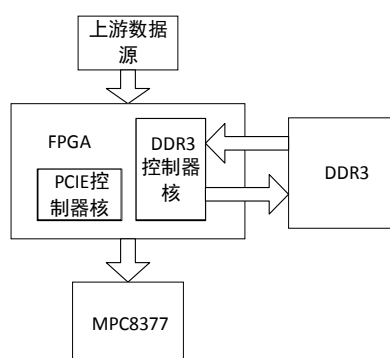


图3.10 FPGA-PowerPC 通信方案结构框图

为满足上述功能要求，最终选定 FPGA 为 Xilinx 公司的 Virtex6 XC6VLX240T。PowerPC 选用 Freescale 公司的 MPC8377 处理器。

在 PCIe 通信系统中，FPGA 主要负责实现 PCIe EP 设备，与 PowerPC 处理器通信，同时为了测试数据传输功能，FPGA 还要负责数据源的生成和缓存。XC6VLX240T 是 Xilinx 公司 Virtex6 家族的 FPGA 器件，具备高性能 FPGA 逻辑单元和先进的 GTX 高速串行收发器，集成功能强大的混合模式时钟管理器（MMCM）和 36Kb block RAM/FIFOs。同时 XC6VLX240T 还集成进行 PCI Express 接口设计所需的块资源，使得它可实现遵循 PCI Express 2.0 协议规范的通信^[24]。通过 GTX 收发器可支持 PCI Express Gen1（2.5Gbps）和 Gen2（5Gbps）两种规范的通信，并具备实现 PCI Express 端点设备和根设备的能力，可支持 x1, x2, x4, x8 lane。XC6VLX240T 所具备的资源 and 性能，完全能够满足 PCIe 通信方案的需求，故选用该器件搭建 FPGA 开发平台。

PowerPC 处理器是该系统的主处理器，是 PCIe 通信的控制者和发起者。初始化和控制 EP 设备、发起 PCIe 通信等操作都由运行在 PowerPC 上的程序完成。MPC8377 是 Freescale 公司推出的 PowerPC 架构处理器，隶属于 PowerQUICC II Pro 系列产品线，能够为互联型设备提供高性价比的解决方案。MPC8377 的工作主频最高可达 800MHz，采用 e300 内核，带有 MMU（内存管理单元），可运行 Linux 操作系统、vxWork 操作系统。片内集成 DDR2 控制器、PCIe 控制器、100M/1000M 以太网接口、SATA 控制器等外设接口^[20]。该处理器能够满足 PCIe 通信功能，并具有多种外设接口进行功能扩展，方便功能测试。

3.2 PCIe 通信方案硬件设计

3.2.1 FPGA 系统硬件设计

FPGA 及其外围电路和器件将以全部装配在一块板卡上，并与 PowerPC 系统板上的 PCIe 插槽连接。板卡上的核心器件是 XC6VLX240T FPGA。FPGA 的 GTX IO 引脚连接到板卡的金手指上，这样就可以通过 PCIe 插槽进行 PCIe 通信。同时整个板卡的电源也全部从 PCIe 插槽的电源引脚获得，PCIe 插槽的供电能力能够达到 75W，可以满足板卡的功率需求。

PCIe 的物理接口是由 FPGA 内部集成的 GTX 高速串行收发器实现。高速串行收发器对时钟的抖动和延迟等参数的要求较高。若时钟由 FPGA 的 MMCM 时钟管理器提供，由于时钟抖动和延迟较大，会导致 PCIe 接口工作不稳定。所以 PCIe 部分的参考时钟信号由 PCIe 插槽提供，时钟信号经过 FPGA 内部的时钟 buffer 后直接输出 PCIe 核。

高速串行收发器还对电源质量有较高的要求。在本系统中,采用多路 PTH03010W DC-DC 电源转换模块将 PCIe 插槽的 3.3V 供电转换为 FPGA 所需的 2.5V、1.5V 和 1.2V,并在 GTX 使用的 2.5V 电源输入处并联 47 μ F 的大容量储能电容,以保证高速串行收发器的稳定工作。

DDR3 SDRAM 以内存条的形式插在板卡上的内存插槽上。FPGA 的 IO 管脚到 DDR3 的映射可由 FPGA 的 DDR3 控制器 IP 核确定,并可以进行自定义。

3.2.2 嵌入式 PowerPC 系统硬件设计

嵌入式 PowerPC 系统主要由 MPC8377 处理器、DDR2 SDRAM、NOR flash 存储器、PCIe 接口和 SATA 接口组成。

其中 MPC8377 处理器是该系统的核心。由 66.666MHz 外部晶振为 MPC8377 提供时钟信号。从 MPC8377 引出一组串口和 JTAG 接口进行调试。由于 MPC8377 的地址总线 and 数据总线是复用的,所以必须在总线上加锁存器。在一次总线操作中,PowerPC 先在总线上打出寻址信号,该信号被地址锁存器锁存后,PowerPC 再在总线上打出数据信号,并由数据锁存器锁存该信号。MPC8377 总线结构如图 3.2。

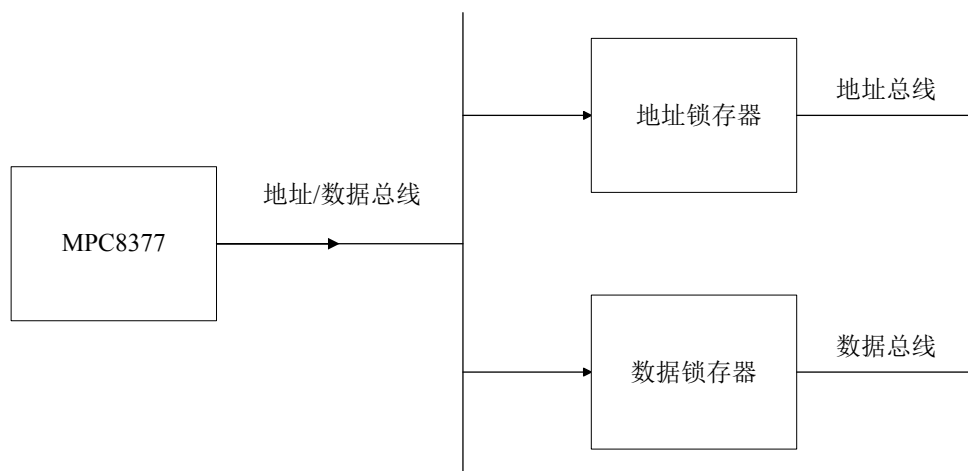


图3.11 MPC8377 的地址/数据总线

PowerPC 系统的 DDR2 SDRAM 选用 4 片镁光公司的 MT47H32M16HW DDR2 SDRAM 颗粒,每片容量为 512Mbit,工作在 x16 位宽模式下,通过数据总线与地址总线与 MPC8377 相连,为系统提供 256MB 的内存空间。

NOR flash 选用 Spansion 公司的 S29GL064N90TFI030 64Mbit flash 存储器,为系统提供 8MB 的非易失存储空间。NOR flash 存储器上主要存放 Linux 系统的 uboot 和内核镜像、PowerPC 的设备结构树,以及 Linux 文件系统。

为方便测试时将从 PCIe 接口收到的数据保存到硬盘上分析,PowerPC 系统还应

支持挂载硬盘。

PCIe、SATA 等高速串行信号的串行串解器 (SerDes) 对参考时钟的质量要求很高, 普通晶振难以满足其要求。所以在该系统中, 选用专用的高性能时钟产生器作为串行串解器参考时钟的时钟源。

ICS841484I 是专为 PCIe 和 sRIO 优化设计的时钟产生器。该器件可由 25MHz 晶体振荡器产生 100MHz、125MHz、200MHz 和 400MHz 的时钟信号, 并可以替代需要外加多个振荡器和时钟扇出驱动器的解决方案。ICS841484I 的相位抖动 (phase jitter) 性能非常出色, 非常适合作为要求高精度低抖动时钟信号的 PCIe 或 sRIO 器件的时钟源。ICS841484I 还支持应用于 PCIe 的可配置的频谱扩展 (spread-spectrum)。该器件适合应用于驱动通用处理器集成的 PCIe 或 sRIO 接口的高速串行串解器时钟, 也适用于 SATA 接口^[25]。

ICS841484I 支持 4 路 0.7V 差分时钟信号输出, 可同时提供 MPC8377 一路 PCIe 和两路 SATA 的串行串解器参考时钟, 并且其时钟输出信号的性能参数也能够满足要求。

MPC8377 嵌入式 PowerPC 处理器片内集成符合 PCIe V1.0a 标准规范的 PCIe 总线控制器。MPC8377 上的 PCIe 总线控制器可以工作在 RC 模式或 EP 模式。PCIe 通信系统方案中, FPGA 作为 EP 设备, 而 MPC8377 的 PCIe 总线控制器工作在 RC 模式, 作为 RC 设备。

MPC8377 的 PCIe 总线物理层是基于片内集成的串行串解器物理层功能块 (SerDes PHY block) 的。MPC8377 的串行串解器物理层功能块包括两组串行串解器, 每组有两个 lane, 其中 SerDes2 支持一路 x2 宽度的 PCIe 链路, 两路 x1 宽度的 PCIe 链路, lane 峰值带宽可达 2.5GT/s, 能提供支持冷复位和热复位的掉电复位状态机和 reset_done 指示, 并带有内部时钟控制器。每组串行串解器都需要一个参考时钟信号, 该信号是基于外部时钟信号在串行串解器内部生产的。MPC8377 的串行串解器物理层功能块框图如图 3.3 所示。

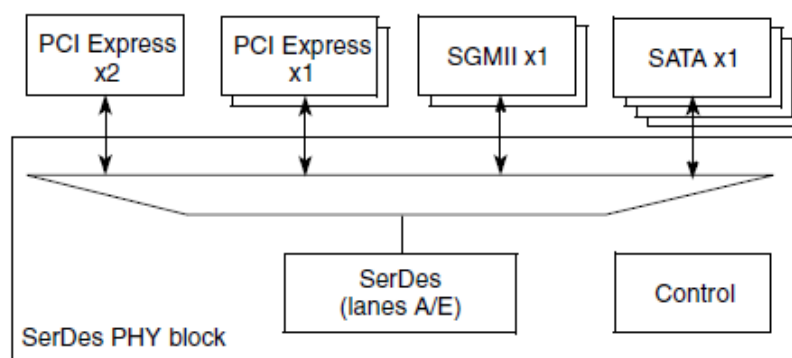


图3.12 MPC8377 的串行串解器物理层功能块框图

一组串行串解器的外部信号线如表 3.1 所示：

表3.2 MPC8377 串行串解器的外部信号

信号	I/O	描述
SD_IMP_CAL_RX	I	接收器阻抗匹配控制信号。该管脚需要经过外部电阻器接地，以设置接收器的差分输入阻抗。
SD_IMP_CAL_TX	I	发送器阻抗匹配控制信号。该管脚需要经过外部电阻器接地，以设置发送器的差分输出阻抗。
SD_REF_CLK	I	串行串解器 PLL 参考时钟正极，PLL 使用这个时钟产生串行串解器需要的所有时钟
#SD_REF_CLK	I	串行串解器 PLL 参考时钟负极，PLL 使用这个时钟产生串行串解器需要的所有时钟
SD_RXA	I	Lane A 的串行数据接收管脚正极
#SD_RXA	I	Lane A 的串行数据接收管脚负极
SD_RXE	I	Lane E 的串行数据接收管脚正极
#SD_RXE	I	Lane E 的串行数据接收管脚负极
SD_TXA	O	Lane A 的串行数据发送管脚正极
#SD_TXA	O	Lane A 的串行数据发送管脚负极
SD_TXE	O	Lane E 的串行数据发送管脚正极
#SD_TXE	O	Lane E 的串行数据发送管脚负极

串行串解器内部集成有效阻抗匹配电路，以保证发送/接收链路的终端电阻的阻抗值控制在最佳状态。匹配电路使用外部连接的电阻对应内部阻抗。对于接收器，用户应在 SD_IMP_CAL_RX 管脚和电源 GND 之间接入精度 1% 的 200 欧姆电阻；对于发送器，用户应在 SD_IMP_CAL_TX 管脚和电源 GND 之间接入精度 1% 的 100 欧姆电阻。

除支持 PCIe 链路外，MPC8377 的串行串解器物理层功能块还支持符合 SATA2.5 规范的总线链路。MPC8377 的 SerDes1 支持两路带宽为 1.5Gbps 或 3.0Gbps 的 SATA 链路。MPC8377 可挂载支持 SATA 接口的机械硬盘或固态硬盘，硬盘通过 SATA 数据线连接在 SATA 插座上，并需要一路 5V 电源为硬盘供电。

3.3 FPGA 的逻辑设计

FPGA 上实现的主要逻辑功能有：数据产生，产生用来测试数据传输功能的数据源；数据缓存，将数据缓存至 DDR3；数据发送，将 DDR3 中缓存的数据通过 PCIe 总线发送。FPGA 逻辑功能框图如图 3.4 所示。

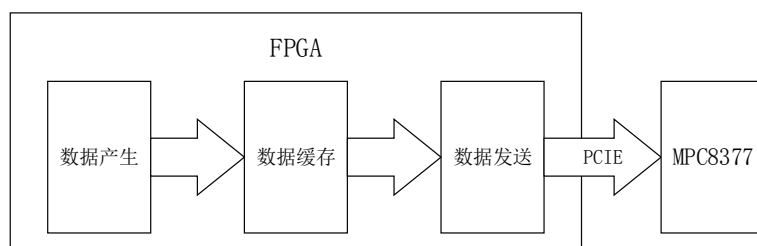


图3.13 FPGA 逻辑功能框图

数据生成模块主要功能是产生数据源，主要组件是一个 ROM 和一个用于同步时钟域和转换数据位宽的 FIFO^[26]。

数据生成模块的 ROM 负责存放数据源，其寻址信号由一个循环计数器产生，对 ROM 中的数据进行循环读取，可产生数据流。ROM 的数据位宽为 16 位，而缓存模块的位宽为 64 位，数据通过输入为 16 位、输出为 64 位的 FIFO 缓存后，即可转换为 64 位宽度。

数据缓存模块的功能可以概括为把外部的 DDR3 SDRAM 作为用来缓存数据的 FIFO。由于后端 PCIe 模块并不是连续读取数据的，每次读取操作之间可能存在长短不一的间隙，所以这个缓存 FIFO 是必要的。数据缓存模块的主要组件有上游数据 FIFO、下游数据 FIFO、DDR3 SDRAM MIG 核，DDR3 FIFO 逻辑。上游数据 FIFO 和下游数据 FIFO 负责将数据的时钟域与 DDR3 MIG 核同步，并将数据位宽转换为与 DDR3 MIG 核接口位宽相同的 256 位。DDR3 MIG 核将 DDR3 的复杂接口和操作时序转换位面向用户的简单接口和时序。DDR3 FIFO 逻辑则把 DDR3 用户接口封装成 FIFO，进一步简化外部逻辑对 DDR3 的操作。数据缓存模块框图如图 3.5 所示。

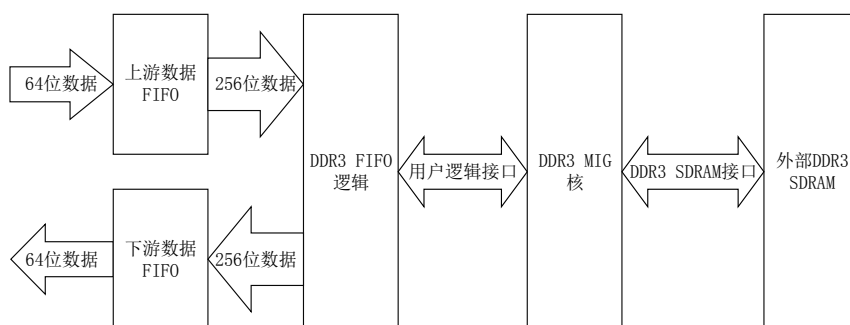


图3.14 数据缓存模块框图

数据发送逻辑由 PCIe 硬核和 PCIe DMA 逻辑引擎组成。PCIe 硬核是支持 Virtex6 系列 FPGA 的集成 IP 核，该 IP 核符合 PCIe v2.0 标准规范，支持 2.5GT/s 和 5GT/s 的链路带宽，可配置为 EP 或 RC。这里将 PCIe 硬核配置成 EP，并设置好链路参数、BAR 空间地址和 PCIe 配置寄存器。FPGA 要实现通过 PCIe 总线进行大量数据传输，还需要 DMA 引擎的支持。DMA 引擎可通过映射在 BAR 空间中的寄存器来操作。DMA 引擎由发送引擎、接收引擎、DMA 控制状态寄存器、中断控制等模块组成，其框图如图 3.6 所示。

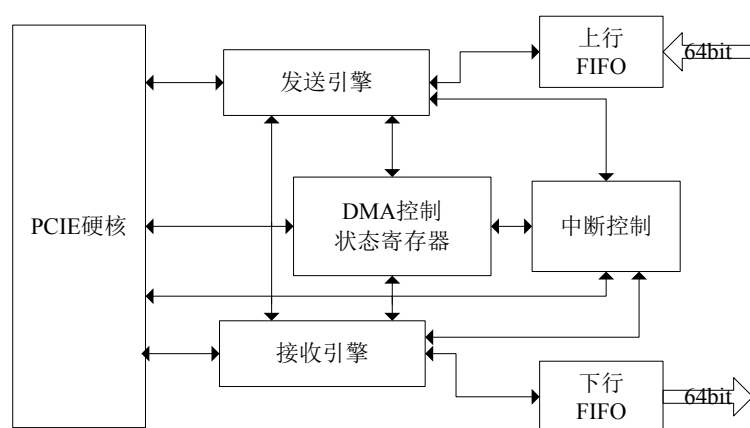


图3.15 数据发送模块框图

RC 上的应用程序在写 PCIe BAR0 空间时，会生成具有相应地址、字节使能和寄存器内容的写请求 TLP，接收引擎根据 PCIe 协议规定的基本 TLP 接收时序，从 PCIe 核接口上获取该写请求 TLP。通过解析写请求 TLP，获取地址和寄存器内容，传递给 DMA 控制状态寄存器部分。DMA 控制状态寄存器根据地址和寄存器内容，完成对 DMA 的配置和操作。

发送引擎会根据 DMA 状态控制寄存器中的写目的地址和写传输长度，按照 PCIe 接口时序，填充写请求 TLP 的相关信息和数据，从而发起 DMA 写请求；如果启动的是 DMA 读请求操作，则发送引擎会根据 DMA 状态控制寄存器中的读源地址和读传输长度，按照 PCIe 接口时序，通过填充读请求 TLP 相关信息，从而发起 DMA 读请求。

接收引擎主要负责接收 RC 发起的读写请求 TLP，并对获取的存储器读写请求 TLP 进行解析，之后将解析得到的部分 TLP 信息传递给发送引擎、DMA 状态控制寄存器和中断控制。在 DMA 读写开始之前，应用程序需要通过写 PCIe BAR0 空间来实现 DMA 状态参数和 DMA 读写中断的配置。

中断控制模块根据 DMA 控制器的状态，产生 DMA 读完成中断或 DMA 写完成

中断，RC 端的驱动程序能捕获这些中断，并调用相应的中断处理函数。

3.4 PowerPC 软件设计

3.4.1 MPC8377 PCIe 控制器配置

(1) 串行串解器功能块配置

MPC8377 有两组 PCIe 控制器，在本系统中只用到控制器 1。为使 MPC8377 的 PCIe 控制器正常工作，首先要在 uboot 中配置串行串解器的工作模式。该配置由串行串解器功能块控制寄存器组的管理，其主要字段定义如表 3.2。

表3.3 串行串解器功能块控制寄存器组需要配置的寄存器字段

寄存器	位	名称	描述
SRDSnCR1	25	PLLBW	选择串行串解器的带宽 0: PCIe 未使能时置 0 1: PCIe 已使能时置 1
SRDSnCR2	19-21	SEICA	Lane A 接收器电气空闲检测控制 100: 默认 PCIe, SATA1 电平标准 101: 默认 SATA2 电平标准
	27-29	SEICE	Lane E 接收器电气空闲检测控制 100: 默认 PCIe, SATA1 电平标准 101: 默认 SATA2 电平标准
SRDSnCR3	6-7	SDFMA	设置数字滤波器带宽以优化 lane A 的频率偏移 00:200ppm 01:600ppm (PCIe、SATA 推荐)
	14-15	SDFME	设置数字滤波器带宽以优化 lane E 的频率偏移 00:200ppm 01:600ppm (PCIe、SATA 推荐)
	21-23	SDTXLA	控制 lane A 发送器的幅值电平 000: 无幅值衰减 (PCIe 推荐) 101: $0.583 \times$ 最大摆幅 (SATA 推荐)
	26-31	SDTXLE	控制 lane E 发送器的幅值电平 000: 无幅值衰减 (PCIe 推荐) 101: $0.583 \times$ 最大摆幅 (SATA 推荐)

SRDSnCR4	2-3	RFCKS	串行串解器参考时钟选择 00: 100MHz 01: 125MHz 11: 150MHz
	7	PLANE	PCIe lane 数量 0: PCIe1 使用 lane A, PCIe2 使用 lane E 1: PCIe1 使用 lane A 和 lane E, PCIe2 不使能
	20-23	PROTA	Lane A 协议选择 0001: PCIe 2.5Gbps 1000: SATA 3.0Gbps 或 SATA 1.5Gbps
	28-31	PROTE	Lane E 协议选择 0001: PCIe 2.5Gbps 1000: SATA 3.0Gbps 或 SATA 1.5Gbps

在 uboot 中对上述寄存器进行修改: SRD2SCR1 的 PLLBW 位设置为 1; SRD2SCR2 的 SEICA 和 SEICE 位设置为 100; SRD2SCR3 设置为 0x01010000; SRD2SCR4 的 RFCKS 位设置为 00, PLANE 位设置为 0, PROTA 和 PROTE 位设置为 0001。这样便将 SerDes2 设置为 PCIe x1 工作模式。再将 SerDes1 设置为 SATA 工作模式。SRD1SCR1 的 PLLBW 位设置为 0; SRD1SCR2 的 SEICA 和 SEICE 位设置为 101; SRD1SCR3 设置为 0x15150505; SRD1SCR4 的 RFCKS 位设置为 00, PLANE 位设置为 0, PROTA 和 PROTE 位设置为 1000。以上寄存器配置完成后, 要对 SRDSnRSTCTL (SerDes n 复位控制寄存器) 的 RST 位设置为 1, 对两组串行串解器各进行一次复位操作。复位完成后, 串行串解器复位状态机会自动清零 RST 位。

(2) PCIe 控制器的时钟配置

PCIe 控制器的时钟配置由系统时钟配置寄存器 (system clock configuration register, SCCR) 管理, 其主要字段定义如表 3.3。

表3.4 系统时钟配置寄存器需要配置的字段

位	名称	描述
10-11	PCieXP1CM	PCie 1 控制器时钟模式 00: PCie 1 时钟不使能 01: PCie 1 时钟:csb_clk=1:1 10: PCie 1 时钟:csb_clk=1:2 11: PCie 1 时钟:csb_clk=1:3
12-13	PCieXP2CM	PCie 1 控制器时钟模式 配置同 PCieXP1CM
24-25	SATA1CM	SATA1 控制器时钟模式 00: SATA1 时钟不使能 01: SATA1 时钟:csb_clk=1:1 10: SATA1 时钟:csb_clk=1:2 11: SATA1 时钟:csb_clk=1:3
26-27	SATA2CM	SATA2 控制器时钟模式 配置同 SATA1CM
28-29	SATA3CM	SATA3 控制器时钟模式 配置同 SATA1CM
30-31	SATA4CM	SATA4 控制器时钟模式 配置同 SATA1CM

通过设置 SCCR 寄存器，将 PCie 和 SATA 的时钟分频都设置成 1:1。

(3) PCie 控制器工作模式配置

作为 PCie 主机，MPC8377 的 PCie 控制器应工作在 RC 设备类型下，而默认的设备类型是 EP，需要在 uboot 中对其进行变更。PCie 控制器的设备类型是由 PECR（PCie 控制寄存器）管理的，其主要字段定义如表 3.4。

表3.5 PCie 控制寄存器需要配置的字段

位	名称	描述
0	LINK_RST	链路软复位（低有效）。 对 PCie 控制器中与链路相关的逻辑进行软复位，在设备类型被改变和串行串解器完成初始化时应进行链路软复位。
1	CBRST	CSB 桥软复位（低有效）。

		对 PCIe 控制器中与 CSB 桥相关的逻辑进行软复位, 在设备类型被改变和串行串解器完成初始化时应进行 CSB 桥软复位。
2	CSRST	CSR 软复位 (低有效)。 对 PCIe 控制器中与 CSR (control and status registers) 相关的逻辑进行软复位, 在设备类型被改变和串行串解器完成初始化时应进行 CSR 软复位。
16	DEV_TYPE	PCIe 设备类型 0: EP (endpoint) 1: RC (root complex)

将 PCIe 设备类型变更为 RC 后, 需要对链路、CSB 桥和 CSR 进行软复位, 将 PECR 寄存器相应的位设置为 0 即可进行复位操作。

(4) 本地访问窗口配置

在访问 PCIe 控制器之前, 要将 PCIe 控制器的内存空间映射至本地总线。这个地址映射是通过本地访问窗口 (local access window) 完成的。本地访问窗口可将外设的外部地址映射为处理器可见的本地总线地址。在定义一个本地访问窗口时, 需要设置两组寄存器: 窗口基地址寄存器 (BAR) 和窗口属性寄存器 (AR)。定义 PCIe1 的本地访问窗口需配置 PCIEXP1LAWBAR (本地访问窗口基地址寄存器) 和 PCIEXP1LAWAR (本地访问窗口属性寄存器), 其主要字段定义如表 3.5 和 3.6。

表3.6 本地访问窗口基地址寄存器需要配置的字段

位	名称	描述
0-19	BASE_ADDR	本地访问窗口基地址的高 20 位。这个基地址应与本地访问窗口大小字节对齐。

表3.7 本地访问窗口属性寄存器需要配置的字段

位	名称	描述
0	EN	本地访问窗口使能
26-31	SIZE	本地访问窗口大小, 窗口大小是 $2^{(SIZE+1)}$ 个字节。窗口大小必须大于 4K 字节, 小于 2G 字节。

在 uboot 中, 将 PCIe1 的总线访问窗口基地址设置为 0xA0000000, 窗口大小设置为 512MB, 并将窗口使能。至此, 访问 PCIe 控制器的准备工作已经完成。

(5) CSB 桥和地址译码窗口的相关寄存器配置

首先对 PCIe 控制器的 CSB 桥进行配置。在 CSB 桥控制寄存器 (PEX_CSB_CTRL) 中要使能 CSB 的 inbound PIO 和 outbound PIO 功能。还要在 CSB 桥 inbound 控制寄存器 (PEX_CSB_IBCTRL) 和 outbound 控制寄存器 (PEX_CSB_OBCTRL) 中使能 PIO, 并使能配置、I/O、内存三个类型的 outbound 窗口, 其主要字段定义如表 3.7、3.8、3.9。

表3.8 CSB 桥控制寄存器需要配置的字段

位	名称	描述
1	IBPIOE	Inbound PIO 使能
0	OBPIOE	Outbound PIO 使能

表3.9 Outbound 控制寄存器需要配置的字段

位	名称	描述
3	CFGWE	Outbound 配置窗口使能
2	IOWE	Outbound I/O 窗口使能
1	MEMWE	Outbound 内存窗口使能
0	PIOE	Outbound PIO 使能

表3.10 Inbound 控制寄存器需要配置的字段

位	名称	描述
0	PIOE	Inbound PIO 使能

将 PCIe 总线地址和 CSB 地址之间的映射, 是由地址译码窗口 (ATMUs) 完成的。PCIe 的总线地址空间分为内存空间、I/O 空间和配置空间。PCIe 内存事务可寻址 32 位或 64 位的地址空间。根据窗口译码地址的不同, PCIe 控制器作为内存事务的发起者, 可以发送 32 位寻址或 64 位寻址的数据包。来自 CSB 的任何译码地址高于 4G 的事务, 都以 64 位寻址的方式发出, 译码地址低于 4G 的, 则以 32 位寻址的方式发出。而来自 PCIe 总线的事务, 无论 32 寻址还是 64 位寻址, 地址都会被译码成 CSB 使用的 32 位地址。作为 I/O 事务的发起者, 所有 I/O 事务都只能访问 32 位地址的 I/O 空间。当 MPC8377 的 PCIe 总线控制器被配置为 RC 设备时, 支持 outbound 配置事务, 但不支持 inbound 配置事务, 作为 EP 设备时则相反。

先配置 CSB 桥的 outbound 地址映射寄存器。这组寄存器管理的是 CSB 内存空间

到 PCIe 总线内存空间的地址映射。这组寄存器是 RC 设备和 EP 设备通用的。MPC8377 的 CSB 桥有 4 个 outbound 窗口，每个 outbound 窗口有 4 组寄存器，分别是窗口属性寄存器 PEX_OWAR、窗口基地址寄存器 PEX_OWBAR、窗口低地址译码寄存器 PEX_OWTLARL 和窗口高地址译码寄存器 PEX_OWTLARH。

表3.11 Outbound 窗口属性寄存器需要配置的字段

位	名称	描述
31-12	SIZE	Outbound 窗口大小的高 20 位，低 12 为固定为 0，单位为字节。
2-1	TYPE	窗口类型 00: 配置 01: I/O 10: 内存
0	EN	使能该窗口

Outbound 窗口属性寄存器的主要字段定义如表 3.10。配置 PEX_OWAR_n 寄存器，将窗口 0 的类型设为配置（CFG），大小为 16MB，并使能；窗口 1 的类型设为内存（memory），大小为 512MB，并使能；窗口 2 的类型设为 I/O，大小为 8MB，并使能。窗口 3 未使用，窗口默认是不使能的，所以无需设置。

表3.12 Outbound 窗口基地址寄存器需要配置的字段

位	名称	描述
31-12	BA	Outbound 窗口基地址（该窗口所对应的 CSB 基地址）的高 20 位，低 12 为固定为 0。

Outbound 窗口基地址寄存器的字段定义如表 3.11。设置 PEX_OWBAR_n 寄存器，将窗口 0 的基地址设为 0xA0000000；将窗口 1 的基地址设为 0xA8000000；将窗口 2 的基地址设为 0xB8000000。

表3.13 Outbound 窗口低地址译码寄存器需要配置的字段

位	名称	描述
31-12	TAL	Outbound 窗口译码地址的低地址部分，该寄存器只能设置 TA[31:12]，TA[11:0] 固定为 0；

表3.14 Outbound 窗口高地址译码寄存器需要配置的字段

位	名称	描述
31-0	TAH	Outbound 窗口译码地址的高地址部分，该寄存器只能设置 TA[64:32]。

Outbound 窗口地址译码寄存器的字段定义如表 3.12 和表 3.13。设置 PEX_OWTARLn 和 PEX_OWTARHn 寄存器，将窗口 0 的译码地址设为 0x0000000000000000；将窗口 1 的译码地址设为 0x00000000A8000000；将窗口 2 的基地址设为 0x0000000000000000。

接下来配置 CSB 桥的 inbound 地址映射寄存器。对于 inbound 窗口，RC 设备和 EP 设备使用不同的两组寄存器来配置，这里我们只修改 RC 的 inbound 地址映射寄存器。这组寄存器管理的是 PCIe 总线内存空间到 CSB 内存空间的地址映射。MPC8377 的 CSB 桥有 4 个 RC inbound 窗口，每个 inbound 窗口有 4 组寄存器，分别是窗口属性寄存器 PEX_RCIWAR、窗口译码地址寄存器 PEX_RCIWTAR、窗口低基地址寄存器 PEX_RCIWBARL 和窗口高基地址寄存器 PEX_RCIWBARH。

表3.15 Inbound 窗口属性寄存器需要配置的字段

位	名称	描述
27-12	SIZE	Inbound 窗口大小的 27-12 位，低 12 为固定为 0，单位为字节。
2-1	TYPE	窗口类型 01：可预取内存。Inbound 读事务对 CSB 进行性能优化。内存读事务的实际地址和大小可能与原始 PCIe 包不同。 11：不可预取内存。Inbound 读事务对 CSB 进行性能优化。来自 PCIe 总线的内存读事务访问的地址和大小不变。该模式未对 CSB 进行性能优化。
0	EN	使能该窗口

Inbound 窗口属性寄存器的字段定义如表 3.14。配置 PEX_RCIWARn 寄存器，将窗口 0 的类型设为可预取，大小为 256MB，并使能；在剩余内存空间足够的情况下，继续使能剩余的窗口。

表3.16 Inbound 窗口译码地址寄存器需要配置的字段

位	名称	描述
31-12	TA	Inbound 窗口译码地址（该窗口所对应的 CSB 基地址）的高 20 位，低 12 为固定为 0。

Inbound 窗口译码地址寄存器的字段定义如表 3.15。配置 PEX_RCITARn 寄存器，将窗口 0 的译码地址设为 0x00000000；其他窗口的译码地址一次递增 256M。

表3.17 Inbound 窗口低基地址寄存器需要配置的字段

位	名称	描述
31-12	BAL	Inbound 窗口基地址的低地址部分，该寄存器只能设置 BA[31:12]，BA[11:0] 固定为 0；

表3.18 Inbound 窗口高基地址寄存器需要配置的字段

位	名称	描述
31-0	BAH	Inbound 窗口基地址的高地址部分，该寄存器只能设置 BA[64:32]。

Inbound 窗口基地址寄存器字段定义如表 3.16 和表 3.17。设置 PEX_OWBARLn 和 PEX_OWBARHn 寄存器，将窗口 0 的基地址设为 0x0000000000000000；其他窗口的基地址依次递增 256M。

ATMUs 设置好后，使能主机的虚拟 INTx 中断。将 CSB 系统中断使能寄存器的 8-5 位设置为 1。CSB 系统中断使能寄存器的字段定义如表 3.18。

表3.19 CSB 系统中断使能寄存器需要配置的字段

位	名称	描述
8	INTDIE	INTD 中断使能，只对 RC 有效。
7	INTCIE	INTC 中断使能，只对 RC 有效。
6	INTBIE	INTB 中断使能，只对 RC 有效。
5	INTAIE	INTA 中断使能，只对 RC 有效。

(6) type1 配置头寄存器配置

配置完以上寄存器后，执行一次 P2P 桥配置操作。这需要更新 type1 配置头寄存器，将 Primary Bus Number、Secondary Bus Number 和 Subordinate Bus Number 设置为 0、1、255。将 Command 的如下位置为 1：0（使能 I/O 空间响应）、1（使能内存空间响应）、2（使能总线主机功能）、6（奇偶校验）、8（使能 SERR）。将 Status 清零（写 1 清零），并将 Latency Timer 设为 0x80，Cache Line Size 设为 0x08。Vendor ID 默认为 0x1957（Freescale），Device ID 默认为 0x00C6（MPC8377E），Vendor ID 和 Device ID 不可修改。PCIe type1 配置头的描述如表 3.19。

表3.20 PCIe type1 配置头描述

名称				地址偏移
Device ID		Vendor ID		00
Status		Command		04
Class Code			Revision ID	08
BIST	Header Type	Latency Timer	Cache Line Size	0C
				10
				14
Secondary Latency Timer	Subordinate Bus Number	Secondary Bus Number	Primary Bus Number	18
Secondary Status		I/O Limit	I/O Base	1C
Memory Limit		Memory Base		20
Prefetchable Memory Limit		Prefetchable Memory Base		24
Prefetchable Base Upper 32Bits				28
Prefetchable Limit Upper 32Bits				2C
I/O Limit Upper 16Bits		I/O Base Upper 16Bits		30
			Capabilities Point	34
Expansion ROM Base Address				38
Brigde Control		Interrupt Pin	Interrupt Line	3C

完成了 type1 配置头的设置后，就可以检查 LTSSM 状态机的状态了。读取 PEX_LTSSM_STAT 寄存器可以获取 LTSSM 状态机的状态码。若状态码为 0x16(L0)，则表示 PCIe 链路已训练完成，进入正常工作模式。至此 uboot 中的 PCIe 初始化工作完成。

3.4.2 EP 设备的驱动程序设计

Linux 应用程序要访问外部设备，必须通过设备驱动程序以文件的形式访问。在 PCIe 通信系统中，也必须为作为 EP 的 FPGA 设计驱动程序。

Linux 系统将设备分为三大类：字符设备、块设备和网络接口^[27]。

字符设备能够以字节流的形式被访问。Linux 操作系统可以对字符设备进行 open()、read()、write()、close()等操作，这与对普通文件的操作相似^[27]。驱动程序必须实现以上这些系统调用。字符设备通过文件系统节点来访问，他与普通文件的主要区别是字符设备通常只能顺序访问。但访问具有数据区特性的字符设备时，也可以前后移动访问的位置^[27]。PCIe 通信系统中的 FPGA 就是这样一种设备。

块设备也是通过文件系统节点来访问，块设备能作为文件系统的载体^[27]。块设备驱动程序和字符设备驱动程序与内核的软件接口是不同的，但 Linux 应用程序对这两种设备的读写操作接口基本相同，所以他们的区别主要在于内核管理数据的方式。但 Linux 应用程序并不关心这些不同。

网络接口可以是硬件设备，也可以是纯软件。任何网络事务都经过一个网络接口形成。网络设备只处理数据包，而不需要关心网络连接，所以它不是面向流的，也不通过文件系统节点访问^[27]。

作为 EP 的 FPGA 驱动程序在逻辑上分为两部分：PCIe 设备初始化和字符型设备驱动。PCIe 设备初始化部分主要调用 Linux 内核中的 PCIe 设备操作 API，设置 PCIe 设备的工作模式，映射内存空间，绑定中断处理函数。字符型设备驱动部分首先初始化字符型设备，并将设备加入设备列表。此外，还要实现设备文件操作的一系列函数，其中较为重要的是 `open()`（打开文件操作）、`release()`（关闭文件操作）、`write()`（写文件操作）、`read()`（读文件操作）、`ioctl()`（I/O 控制操作）。这些 API 是用户访问设备的软件接口，也是驱动程序设计的关键部分。`open()`和 `release()`函数主要完成设备的初始化和资源释放。用户与设备之间的数据交换则主要由 `write()`函数、`read()`函数和 `ioctl()`完成。

PCIe 总线继承了 PCI 总线的许多设计，在软件层面上，二者也有很多兼容之处。一般的 PCIe 设备可以使用 PCI 设备的 API 进行驱动程序设计。

（1）驱动程序中的结构体定义

要在驱动程序中初始化 PCIe 设备，首先要实现 `pci_driver` 结构体，它在 `linux/pci.h` 中声明。该结构体定义 PCIe 设备的名称、类型、探测函数和卸载函数。`pci_driver` 结构体的实现如下：

```
static struct pci_driver pciecard_pci_driver = {
    .name          = "pciecard",
    .id_table       = pciecard_pci_tbl,
    .probe          = pciecard_probe,
    .remove         = __devexit_p(pciecard_remove),
};
```

`pci_driver.name` 成员定义设备名称，由用户自行定义。

`pci_driver.id_table` 成员应以驱动程序可以支持的设备类型，他是一个 `pci_device_id` 类型的结构体数组。`pci_device_id` 结构体包含如下字段：

```
__u32 vendor;
__u32 device;
```

```
__u32 subvendor;
```

```
__u32 subdevice;
```

以上四个字段指定驱动程序所支持的 PCIe 设备的厂商 ID, 设备 ID, 子厂商 ID 和子设备 ID。通过这几种 ID 可以辨认出不同的 PCIe 设备, 以便找到对应的驱动程序。如果驱动程序对符合某个 ID 的所有设备都支持, 这个 ID 的字段应该被赋值为 PCI_ANY_ID^[28]。

```
__u32 class;
```

```
__u32 class_mask;
```

以上两个字段指定驱动程序支持某一类型的 PCIe 设备。这个类型 ID 在 PCIe 规范中描述, 如果驱动程序并不针对某类型的设备, 该字段应该被赋值为 PCI_ANY_ID。

pciecard_pci_tbl 数组的实现如下:

```
static struct pci_device_id pciecard_pci_tbl [] __initdata = {
    {PCIeCARD_VENDOR_ID, PCIeCARD_DEVICE_ID, PCI_ANY_ID, \
    PCI_ANY_ID, PCI_ANY_ID, PCI_ANY_ID, 0},
    {0,}
};
```

其中的 PCIeCARD_VENDOR_ID 是 0x10EE (Xilinx), PCIeCARD_DEVICE_ID 是 0x0007, 是 FPGA PCIe 核中定义的 EP 设备 ID。数组的第二个成员是 {0,}, 表示数组结束。

(2) 探测函数设计

pci_driver.probe 是指向 PCIe 探测函数的指针, 执行装载驱动的操作后, PCIe 设备在 Linux 中被探测到并初始化时, 需要调用该函数。探测函数的实现如下:

```
static int __devinit pciecard_probe(struct pci_dev *pdev, const struct pci_device_id
*pci_id)
{
    u16 vendor_id;
    u16 device_id;
    u32 base_addr_0;
    u32 base_addr_1;
```

以上语句声明一些变量, 用来存储从 PCIe 设备获取的厂商 ID、设备 ID、BAR0 空间基地址和 BAR1 空间基地址。

```
if(pci_enable_device(pdev)){
```

```

        printk(KERN_WARNING "pciecard not enabled\n");
        return -EFAULT;
    }

```

激活 PCIe 设备，若失败，则打印错误信息并返回。

```
pci_set_master(pdev);
```

将设备设置为主模式，这样设备就有了获取总线使用权的能力。因为 EP 要向 MPC8377 发起数据传输，所以需要获取 PCI 总线。

```

if(pci_request_regions(pdev, "pciecard")){
    printk(KERN_WARNING "pci_request_regions failed\n");
    return -EFAULT;
}

```

为 PCIe 设备申请内存空间。

```

hdwr_base_bar0 = pci_resource_start(pdev, 0);
hdwr_end_bar0 = pci_resource_end(pdev, 0);
len_bar0 = pci_resource_len(pdev, 0);
hdwr_base_bar1 = pci_resource_start(pdev, 1);
hdwr_end_bar1 = pci_resource_end(pdev, 1);
len_bar1 = pci_resource_len(pdev, 1);

```

获取 BAR0 空间和 BAR1 空间的首地址、尾地址和长度。获取到的信息保存在全局变量当中。

```

virt_base_bar0 = ioremap(hdwr_base_bar0, len_bar0);
virt_base_bar1 = ioremap(hdwr_base_bar1, len_bar1);

```

将 BAR0 空间和 BAR1 空间映射到虚拟内存空间，Linux 系统所能操作的地址都是虚拟地址。映射后的首地址保存在全局变量当中。

```

pci_read_config_word(pdev, PCI_VENDOR_ID, &vendor_id);
pci_read_config_word(pdev, PCI_DEVICE_ID, &device_id);
printk(KERN_INFO "vendor id 0x%x, device id 0x%x\n", vendor_id, device_id);

```

获取并打印 PCIe 设备的厂商 ID 和设备 ID。

```

        if(request_irq(pdev->irq, pciecard_interrupt, IRQF_DISABLED, "pciecard",
NULL)){
            printk(KERN_WARNING "irq request failed");
            return -EFAULT;
        }
        printk(KERN_INFO "irq request done\n");

```

为 PCIe 设备申请中断，并绑定中断处理函数。

```

tx_bufs_dma = dma_mem_alloc(DMA_SIZE);
rx_bufs_dma = dma_mem_alloc(DMA_SIZE);
tx_bufs = pci_alloc_consistent(pdev, DMA_SIZE, tx_bufs_dma);
rx_bufs = pci_alloc_consistent(pdev, DMA_SIZE, rx_bufs_dma);
printk(KERN_INFO "DMA mapped\n");

```

为 PCIe 设备分配 DMA 缓冲区。

以上是探测函数的 PCIe 初始化部分。下面进行字符设备的初始化。

```

cdev_init(&pciecard_cdev, &pciecard_fops);
register_chrdev_region(pciecard_dev_no, 1, "pciecard");
pciecard_cdev.cdev.owner = THIS_MODULE;
if(cdev_add(&pciecard_cdev, pciecard_dev_no, 1)){
    printk(KERN_WARNING "cdev_add error\n");
    return -EFAULT;
}
printk(KERN_INFO "pciecard_cdev online\n");
return 0;
}

```

以上语句完成了字符设备的初始化和注册，并添加到设备列表中。其中 pciecard_dev_no 是 MKDEV(100, 0)的返回值，即定义设备的主设备号为 100，次设备号为 0。pciecard_fops 是定义设备文件操作的函数指针结构体。

(3) 卸载函数设计

pci_driver.remove 的操作与探测函数相反，在卸载设备前释放资源，实现如下：

```

static int __devexit pciecard_remove(struct pci_dev *pdev)
{

```



```
printk(KERN_INFO "remove\n");
```

```
free_irq(pdev->irq, NULL);
```

释放中断。

```
pci_free_consistent(pdev, DMA_SIZE, rx_bufs, rx_bufs_dma);
```

```
pci_free_consistent(pdev, DMA_SIZE, tx_bufs, tx_bufs_dma);
```

```
rx_bufs = NULL;
```

```
tx_bufs = NULL;
```

```
rx_bufs_dma = NULL;
```

```
tx_bufs_dma = NULL;
```

释放 DMA 缓冲区。

```
unregister_chrdev_region(pciocard_dev_no, 1);
```

反注销字符设备。

```
iounmap(virt_base_bar0);
```

```
iounmap(virt_base_bar1);
```

```
pci_release_regions(pdev);
```

```
virt_base_bar0 = NULL;
```

```
virt_base_bar1 = NULL;
```

释放 BAR0 空间和 BAR1 空间内存。

```
pci_disable_device(pdev);
```

关闭 PCIe 设备。

```
cdev_del(&pciocard_cdev.cdev);
```

删除字符设备。

```
printk(KERN_INFO "pciocard_cdev offline\n");
```

```
return 0;
```

```
}
```

(4) 文件操作定义

在 pciocard_fops 结构体中作如下定义：

```
static struct file_operations pciecard_fops = {
    .owner    = THIS_MODULE,
    .open     = pciecard_open,
    .release  = pciecard_release,
    .unlocked_ioctl    = pciecard_ioctl,
    .write    = pciecard_write,
    .read     = pciecard_read,
};
```

其中 open 和 release 是打开文件和关闭文件操作函数，在本驱动程序中没有特殊的操作。

ioctl 函数在 2.6.36 版本的 Linux 内核中彻底被移除，可以使用 unlocked_ioctl 代替。在 unlocked_ioctl 函数中，定义了设置 PCIe 设备的写 DMA 地址的操作：

```
static int pciecard_ioctl(struct file *file, unsigned int cmd, unsigned long args)
{
    unsigned int addr_phys;
    switch(cmd){
    case CONFIG_WR_DMA_ADDR:
        addr_phys = virt_to_bus(rx_bufs);
        writel(addr_phys, virt_base_bar0 + 0x04);
        writel(0, virt_base_bar0 + 0x08);
```

在 FPGA PCIe EP 设备的 BAR0 空间定义了如表 3.20 和表 3.21 所示的寄存器：

表3.21 DMA 写目的低地址寄存器

位	名称	描述
31-0	DMAWAD_L	DMA 写目的地址低 32 位

表3.22 DMA 写目的高地址寄存器

位	名称	描述
31-0	DMAWAD_H	DMA 写目的地址高 32 位

此处设置这两个寄存器，作用是指定 DMA 的写目的地址到分配好的缓冲区起始地址。主要这两个寄存器接收的地址是 PCIe 总线物理地址，而不是 Linux 操作系统使用的虚拟地址。

```
break;
```

```

default:
    return -EINVAL;
}
return 0;
}

```

write 函数的实现如下:

```

static int pciecard_write(struct file *filp, const char __user *buf, size_t count, loff_t
*offset)
{
    unsigned long p = *offset;
    if(count == 1){
        copy_from_user((virt_base_bar0 + p), buf, 1);
    }

```

向设备的 BAR0 空间写入一个字节。其中 p 是偏移量, p 的值由文件移动文件操作位置的函数 lseek 改变。由于驱动程序工作在内核空间, 而调用驱动程序的应用程序运行在用户空间, 所以应用程序向设备传递数据时要调用 copy_from_user 函数从用户空间向内核空间拷贝数据。

```

    else if(count == 4){
        unsigned long w_reg;
        copy_from_user(&w_reg, buf, 4);
        writel(w_reg, virt_base_bar0 + p);
    }

```

向设备文件写入四个字节。其中 p 是偏移量。这里不直接使用 copy_from_user 将数据写入 BAR0 空间, 而是先将四个字节保存到 32 位变量, 再调用 writel 写入设备 BAR0 空间。如果直接调用 copy_from_user, 会因为 CPU 和 PCIe 总线的大小端不一致而导致数据出错。

```

    else{
        printk(KERN_WARNING "invalid arg count\n");
        return 1;
    }

```

这里写操作只支持每次 1 字节或每次 4 字节。

```

    return 0;
}

```

read()函数的实现如下:

```
static int pciecard_read(struct file *filp, char __user *buf, size_t count, loff_t *offset)
{
    unsigned long p = *offset;
    if(p == READ_DMA_BUF){
        printk(KERN_INFO "s\n");
        wait_event_interruptible(wq, (readl(virt_base_bar0 + 0x28) & 0x00000002)
== 0x00000002);
        printk(KERN_INFO "a\n");
        copy_to_user(buf, rx_bufs, count);
    }
}
```

read()函数的主要功能是从 DMA 缓存区中读取数据, 拷贝到用户空间。读 DMA 缓存区的操作必须在一次 DMA 传输完成后才能进行, 否则有“脏读”的风险。在 FPGA PCIe EP 设备的 BAR0 空间定义了如表 3.22 所示的寄存器:

表3.23 DMA 控制寄存器需要操作的字段

位	名称	描述
0	DMAWS	DMA 写开始控制, 写 1 启动 DMA 写操作, 写 DMA 操作完成后由自动清零
1	DMAWC	DMA 写操作完成后自动置 1, 下次 DMA 写操作开始前必须写 1 清零
2	DMARS	DMA 读开始控制, 写 1 启动 DMA 读操作, 读 DMA 操作完成后由自动清零
3	DMARC	DMA 读操作完成后自动置 1, 下次 DMA 读操作开始前必须写 1 清零

当进行 READ_DMA_BUF 操作时, 驱动程序将挂起, 直至 EP 设备的 DMA 写操作完成, 产生中断并唤醒驱动程序, 驱动程序检查 DMAWC 位, 确认 DMA 写操作完成, 也就是数据已全部写入到驱动程序分配的 DMA 缓冲区中, 再调用 copy_to_user 将 DMA 缓存区的数据传递给应用程序。

```
else if(count == 1){
    copy_to_user(buf, (virt_base_bar0 + p), 1);
}
else if(count == 4){
    unsigned long r_reg;
```

```

        r_reg = readl(virt_base_bar0 + p);
        copy_to_user(buf, &r_reg, 4);
    }

```

该驱动程序也支持以每次 1 字节或每次 4 字节的方式读取 BAR0 空间的寄存器值。

```

    else{
        return 1;
    }
    return 0;
}

```

在探针函数中绑定的中断处理函数实现如下：

```

static irqreturn_t pciecard_interrupt(int irq, void *dev_id, struct pt_regs *regs)
{
    writel(0x00000001, virt_base_bar0 + 0x24);
    printk(KERN_INFO "**** interrupt ****\n");
    wake_up_interruptible(&wq);
    return IRQ_HANDLED;
}

```

在 FPGA PCIe EP 设备的 BAR0 空间定义了如表 3.23 所示的寄存器：

表3.24 DMA 中断状态寄存器关键字段

位	名称	描述
0	DMAWINT	DMA 写完成中断状态，高有效，写 1 清零
1	DMARINT	DMA 读完成中断状态，高有效，写 1 清零

DMA 写完成并产生中断，进入中断处理函数。中断处理函数对 DMAWINT 写 1，将中断状态清除，并唤醒挂起的驱动程序。

（5）装载与卸载接口

实现了驱动函数的基本功能后，还需要为它提供装载与卸载的接口：

```

static int __init pciecard_init_module(void)
{

```

```

    printk(KERN_INFO "init\n");
    pci_register_driver(&pciecard_pci_driver);
    return 0;
}

static void __exit pciecard_cleanup_module(void)
{
    printk(KERN_INFO "exit\n");
    pci_unregister_driver(&pciecard_pci_driver);
}

```

```

module_init(pciecard_init_module);
module_exit(pciecard_cleanup_module);

```

这样作为 EP 的 FPGA 驱动程序就基本完成了。将驱动程序源代码编译成内核模块，以便在 Linux 系统运行时加载该驱动。

3.4.3 RC 端应用程序设计

RC 端应用程序是基于运行在 MPC8377 上的嵌入式 Linux 操作系统的。RC 端应用程序的主要功能是通知 PCIe EP 设备发起 DMA 写操作，读取 DMA 缓冲区中的数据，并将数据写入硬盘。其操作流程如图 3.7 所示。

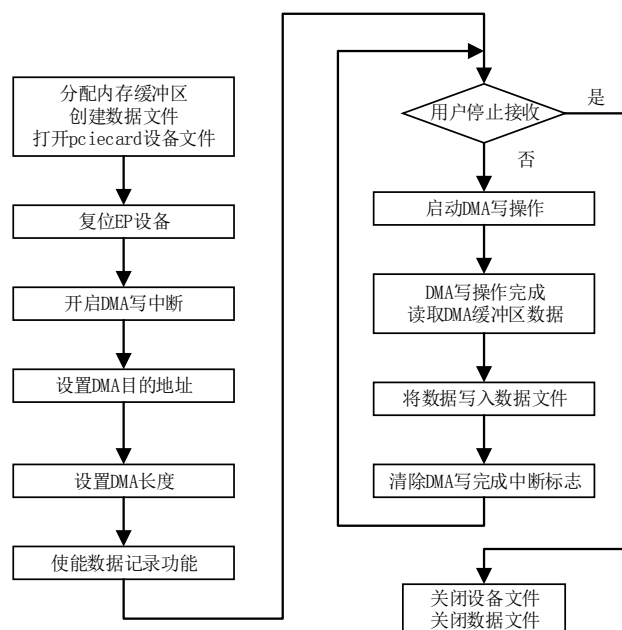


图3.16

RC 端应用程序流程图

在代码中，首先分配内存缓冲区、创建数据文件、打开 pciecard 设备文件。其中数据文件的路径位于系统挂载的硬盘设备上，而 pciecard 设备文件在/dev/目录下：

```
int fd, fd_rx;
unsigned int w_reg, r_reg;
char *dma_addr = NULL;
dma_addr = (char*)malloc(512 * 1024);
if(dma_addr == NULL)
    return 1;
fd_rx = open("/mnt/hdd/rx", O_RDWR | O_CREAT);
fd = open("/dev/pciecard", O_RDWR);
```

对 EP 设备进行复位。复位可以通过操作 EP 设备的软复位寄存器完成，其定义如表 3.24：

表3.25 软复位寄存器需要操作的字段

位	名称	描述
0	SRST	1：施加软复位，0：解除软复位

对 SRST 写 1，再清零，完成软复位：

```
w_reg = 0x00000001;
lseek(fd, 0x2c, 0);
write(fd, &w_reg, 4);
w_reg = 0x00000000;
lseek(fd, 0x2c, 0);
write(fd, &w_reg, 4);
```

EP 设备的中断默认是全部被屏蔽的，要使用中断功能，需要设置中断屏蔽寄存器，其定义如表 3.25：

表3.26 中断屏蔽寄存器关键字段

位	名称	描述
0	WINTMASK	DMA 写完成中断屏蔽，高有效
1	RINTMASK	DMA 读完成中断屏蔽，高有效
31	GINTMASK	全局中断屏蔽，高有效

开启全局中断和 DMA 写完成中断，屏蔽 DMA 读完成中断：

```
w_reg = 0x7fffffe;
lseek(fd, 0x20, 0);
write(fd, &w_reg, 4);
```

设置 DMA 目的地址，该操作是在 ioctl()函数中完成的：

```
ioctl(fd, CONFIG_WR_DMA_ADDR);
```

设置 DMA 长度寄存器，其定义如表 3.26：

表3.27 DMA 写长度寄存器

位	名称	描述
31-0	DMAWXS	DMA 写传输长度。EP->RC 方向，只支持 max_payload_size*2^n

将 DMA 写长度设置为 256K：

```
w_reg = (256*1024);
lseek(fd, 0x18, 0);
write(fd, &w_reg, 4);
```

FPGA 的数据发送模块可以实现数据记录和数据回放两种功能的，记录功能是将数据从 EP 发送至 RC，而回放功能则相反。设置功能控制寄存器，使能记录功能，其定义如表 3.27：

表3.28 功能控制寄存器关键字段

位	名称	描述
0	RCDE	记录使能，高有效
1	REPE	回放使能，高有效

使能 EP 的记录功能：

```
w_reg = 0x00000001;
lseek(fd, 0x38, 0);
write(fd, &w_reg, 4);
```

对 EP 的设置完成后，就可以进行 DMA 传输了。操作 DMA 控制寄存器，将 DMAWS 位置为 1，启动 DMA 传输：

```
lseek(fd, 0x28, 0);
read(fd, &r_reg, 4);
w_reg = r_reg | 1;
```



```
lseek(fd, 0x28, 0);
write(fd, &w_reg, 4);
```

在 DMA 写操作为完成时，进程会挂起，直到 DMA 写操作完成后，中断处理函数将进程唤醒。进程唤醒后应用程序先将 DMA 数据移至用户空间的内存，再写入硬盘上的数据文件：

```
lseek(fd, READ_DMA_BUF, 0);
read(fd, dma_addr, 256*1024);
write(fd_rx, dma_addr, 256*1024);
write(fd, 0x28, 0x00000002);
```

用户结束接收操作后，关闭设备文件和数据文件：

```
close(fd_rx);
free(dma_addr);
return 0;
```

3.5 PCIe 通信功能测试

本节将主要对 PCIe 通信系统的 PCIe 通信功能进行测试。

测试前先将 MPC8377 的调试串口通过串口线与计算机相连。连接好系统的电源线，在计算机的控制台软件中打开调试串口。以上准备工作完成后，为系统上电。首先可以看到系统打印出的 uboot 启动信息，如图 3.8。

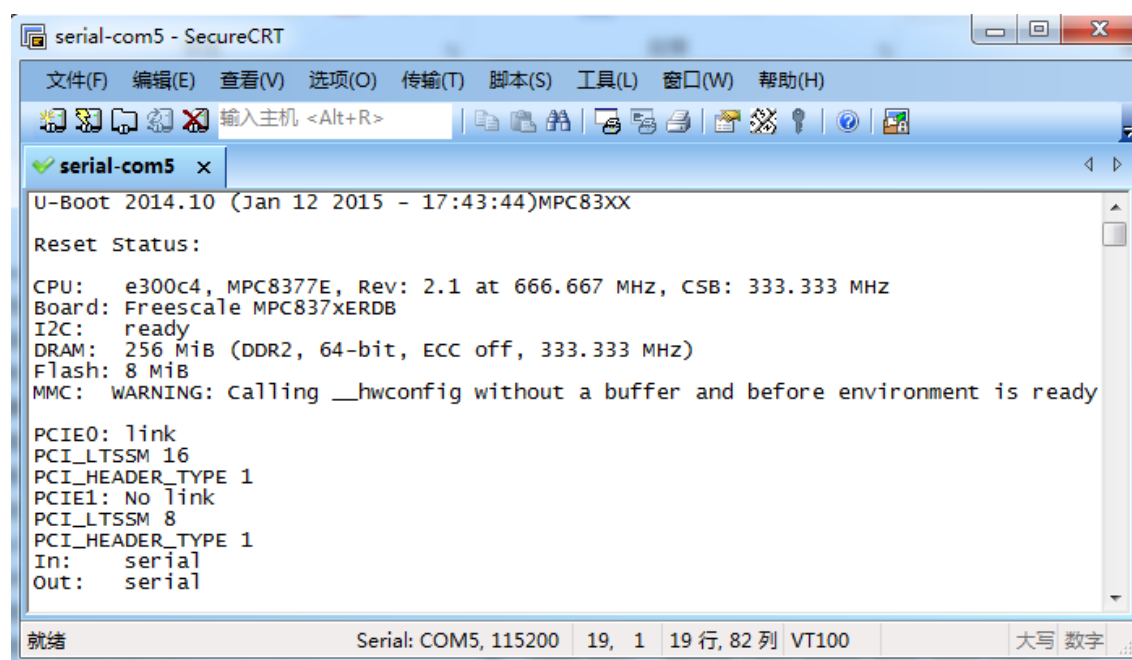


图3.17 系统启动信息

在打印出的信息中可以看到，PCIe 控制器 1（PCIe0）的 LTSSM 状态机已经处于 16（L0）状态，说明 PCIe 链路已经正常建立。配置头类型为 type 1，设备类型为 RC 设备。

在 Shell 输入 `lspci` 命令，查看 PCI/PCIe 总线上的设备，可以看到 EP 设备已被正确识别，如图 3.9。

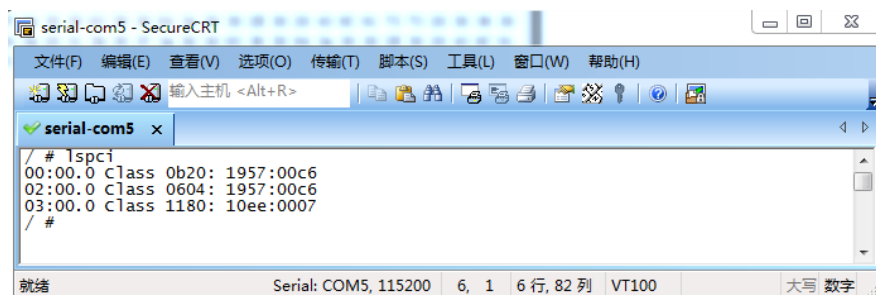


图3.18 PCI 设备列表

将编译好的驱动程序模块复制到文件系统中，用 `insmod` 命令挂载，并用 `mknod` 命令创建设备节点，命令执行完毕后，即可在文件系统的 `/dev` 目录下看到 `pciecard` 设备文件节点，如图 3.10。

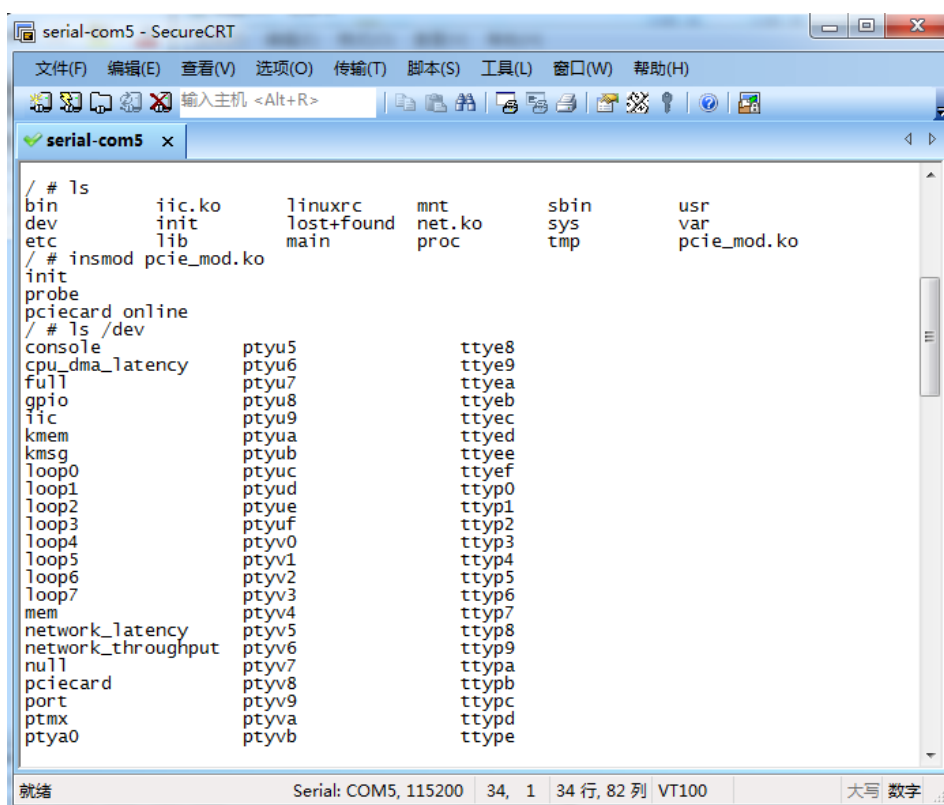
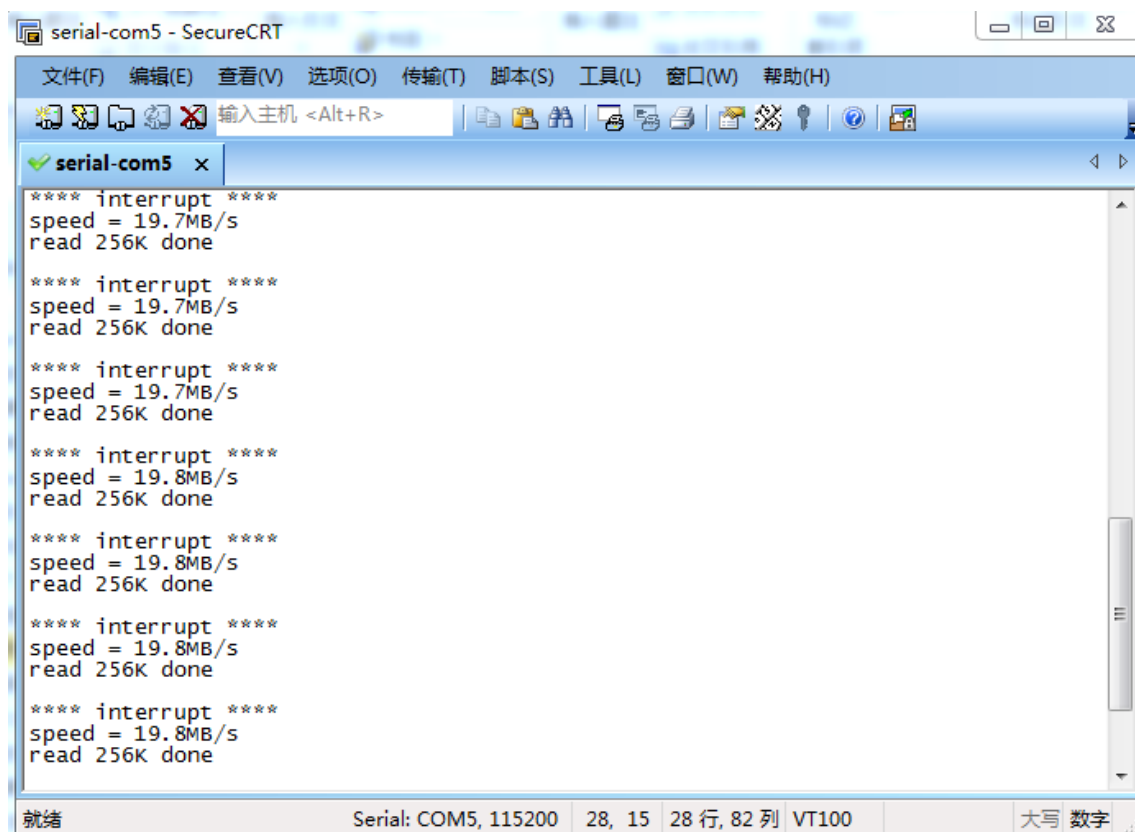


图3.19 装载设备驱动

运行 RC 端应用程序，程序完成初始化工作后会进入主循环，发起 DMA 操作，读取缓冲区中的数据并写入硬盘。

经实际测试，数据的平均传输速度在 20MB/s 左右，没有达到 PCIe 1.0 x1 的正常带宽，这主要是受到硬盘写入速度的限制和写入操作间隔的影响。程序运行时的调试信息如图 3.11 所示。



```
serial-com5 - SecureCRT
文件(F) 编辑(E) 查看(V) 选项(O) 传输(T) 脚本(S) 工具(L) 窗口(W) 帮助(H)
输入主机 <Alt+R>
serial-com5 x
**** interrupt ****
speed = 19.7MB/s
read 256K done

**** interrupt ****
speed = 19.7MB/s
read 256K done

**** interrupt ****
speed = 19.7MB/s
read 256K done

**** interrupt ****
speed = 19.8MB/s
read 256K done

**** interrupt ****
speed = 19.8MB/s
read 256K done

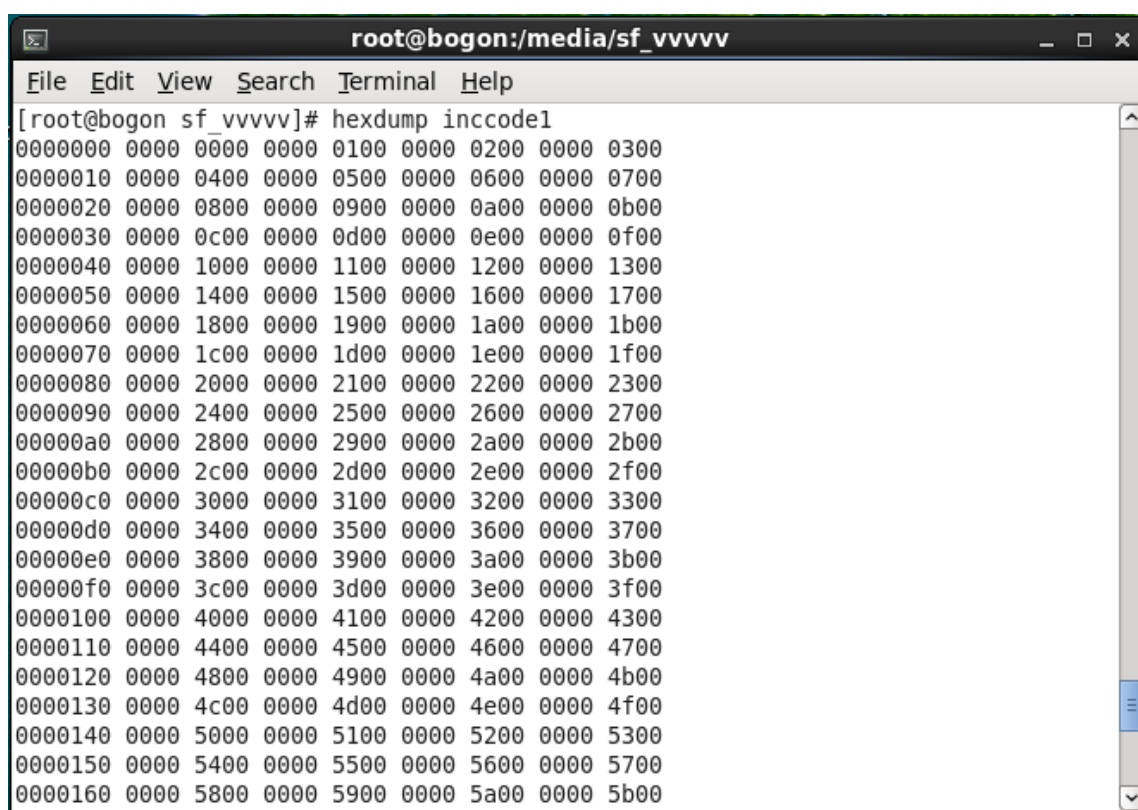
**** interrupt ****
speed = 19.8MB/s
read 256K done

**** interrupt ****
speed = 19.8MB/s
read 256K done

就绪 Serial: COM5, 115200 28, 15 28 行, 82 列 VT100 大写 数字
```

图3.20 PCIe 通信应用程序运行情况

测试时 FPGA 产生并发送的数据为 32 位递增码，接收到的数据经验证完全正确无误。接收到了数据如图 3.12 所示。



A terminal window titled 'root@bogon:/media/sf_vvvvv' with a menu bar (File, Edit, View, Search, Terminal, Help). The command '[root@bogon sf_vvvvv]# hexdump inccode1' has been executed, resulting in a hex dump of 16 lines. Each line contains 16 hexadecimal values, grouped by four spaces. The values represent a sequence of data where the first two bytes of each group are constant (0000) and the last two bytes increment by 0100 for each subsequent group.

```
root@bogon:/media/sf_vvvvv
File Edit View Search Terminal Help
[root@bogon sf_vvvvv]# hexdump inccode1
00000000 0000 0000 0000 0100 0000 0200 0000 0300
00000010 0000 0400 0000 0500 0000 0600 0000 0700
00000020 0000 0800 0000 0900 0000 0a00 0000 0b00
00000030 0000 0c00 0000 0d00 0000 0e00 0000 0f00
00000040 0000 1000 0000 1100 0000 1200 0000 1300
00000050 0000 1400 0000 1500 0000 1600 0000 1700
00000060 0000 1800 0000 1900 0000 1a00 0000 1b00
00000070 0000 1c00 0000 1d00 0000 1e00 0000 1f00
00000080 0000 2000 0000 2100 0000 2200 0000 2300
00000090 0000 2400 0000 2500 0000 2600 0000 2700
000000a0 0000 2800 0000 2900 0000 2a00 0000 2b00
000000b0 0000 2c00 0000 2d00 0000 2e00 0000 2f00
000000c0 0000 3000 0000 3100 0000 3200 0000 3300
000000d0 0000 3400 0000 3500 0000 3600 0000 3700
000000e0 0000 3800 0000 3900 0000 3a00 0000 3b00
000000f0 0000 3c00 0000 3d00 0000 3e00 0000 3f00
00000100 0000 4000 0000 4100 0000 4200 0000 4300
00000110 0000 4400 0000 4500 0000 4600 0000 4700
00000120 0000 4800 0000 4900 0000 4a00 0000 4b00
00000130 0000 4c00 0000 4d00 0000 4e00 0000 4f00
00000140 0000 5000 0000 5100 0000 5200 0000 5300
00000150 0000 5400 0000 5500 0000 5600 0000 5700
00000160 0000 5800 0000 5900 0000 5a00 0000 5b00
```

图3.21 PowerPC 接收到的递增码数据

第四章 多个 PowerPC 处理器的 PCIe 通信设计

随着嵌入式技术的飞速发展, PCIe 设备不再局限于单一功能。在某些应用场合, 要求 EP 设备具有一些操作系统、文件系统的高级特性。如果使用 FPGA 实现这些特性, 不仅开发难度较大, 性能也不及更加廉价的嵌入式处理器。多个嵌入式处理器的 PCIe 通信技术, 在高性能嵌入式平台中有着广阔的应用空间, 对这种技术的研究很有必要。

本文已经对嵌入式 PowerPC 处理器作为 RC 设备做了研究, 并用 MPC8377 处理器实现了 RC 功能。本章将在之前工作的基础上, 进一步研究多片 MPC8377 处理器之间的 PCIe 通信。

目前该方案已应用于网络视频电子存储阵列当中, 其中 RC 接收来自以太网接口的视频数据, 并通过 PCIe 接口转发给 EP, 再由 EP 将数据写入硬盘。

4.1 PCIe 通信方案总体设计

在该 PCIe 通信系统中, 能够建立 2 到 11 个 MPC8377 嵌入式 PowerPC 处理器之间的 PCIe 链路。其中一颗处理器作为 RC 设备, 其他处理器作为 EP 设备。要建立一个 RC 设备对应多个 EP 设备的 PCIe 链路, 需要使用 PCIe switch 进行链路扩展。RC 的 PCIe 接口连接至 switch 的上游端口, 而 EP 的 PCIe 接口连接至 switch 的下游端口。每个 EP 可挂载两块硬盘对数据进行存储。多 PowerPC 通信方案结构框图如图 4.1。

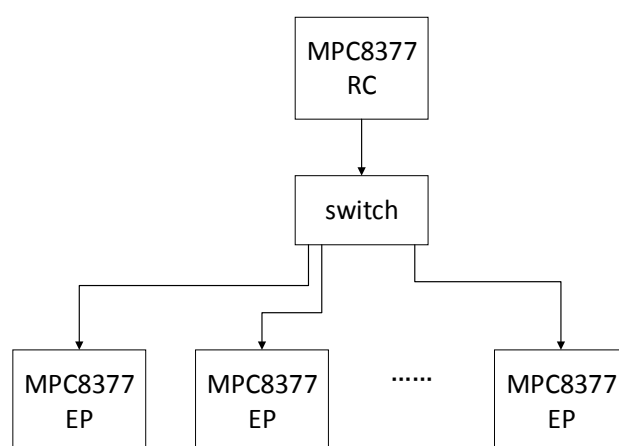


图4.22

多 PowerPC 通信方案结构框图

每颗 MPC8377 处理器都基于独立的最小系统运行。为了存储传输的数据, 使 PCIe 功能测试结果更加直观, MPC8377 系统还应支持挂载硬盘的功能。

4.2 PCIe 通信方案硬件设计

4.2.1 PowerPC 部分硬件设计

PCIe 通信系统的 RC 和 EP 都是以 PowerPC 架构嵌入式处理器 MPC8377 为核心的。其 PowerPC 最小系统部分与第三章所采用的方案基本相同。不同之处在于，RC 要通过 PCIe 总线连接多个 EP，所以加入了 PCIe switch，每个 EP 都需要 PCIe 参考时钟，所以增加了时钟扇出缓冲器提供多路时钟输出。

ICS9DB803DGILFT 是 IDT 公司的差分时钟扇出缓存器，可应用于 PC 或嵌入式系统当中，为系统提供低抖动、低歪斜的 PCIe 时钟信号。ICS9DB803DGILFT 接收一组差分时钟输入，最大支持 8 组差分时钟输出。在 PLL 模式下支持 50~100MHz 的时钟信号输出，在旁路模式下则支持 50~400MHz 的时钟输出^[30]。系统使用两片 ICS9DB803DGILFT 提供 13 组 PCIe 参考时钟信号，其中 2 组输出到 switch，1 组输出到作为 RC 的 PowerPC，10 组输出到作为 EP 的 PowerPC。

4.2.2 PCIe switch 部分硬件设计

为支持多个 EP 设备与 RC 进行 PCIe 通信，需要在 PCIe 链路中加入 PCIe switch，对 PCIe 总线进行链路扩展。PCIe 通信系统采用 89HPES24NT24G2 作为 switch，连接 RC 与 EP 的 PCIe 接口。89HPES24NT24G2 是 IDT 公司的一款 PCIe switch 解决方案，面向 PCIe Gen2 分组交换的高性能应用。最多支持 24 个 lane 和 24 个 PCIe 端口(port)。能满足多主机或基于智能输入/输出域间通信系统的应用需求，可应用于服务器、存储、通信和嵌入式系统中。

89HPES24NT24G2 的 24 个端口可以组合工作，形成 x1、x2、x4 或 x8 宽度的链路。在 PCIe 通信系统方案中，RC 和 EP 的 PCIe 链路宽度均为 x1，即每片 MPC8377 的 PCIe 接口只与 switch 的一个端口相连，共占用 11 个端口。选用 89HPES24NT24G2 的端口 0 作为上游端口，连接作为 RC 的 PowerPC；端口 6 到端口 15 作为下游端口，连接作为 EP 的 PowerPC。每个 PCIe x1 链路由两对差分信号线 RX 和 TX 连接。

89HPES24NT24G2 的 Master SMBus 接口连接一片 24LC512 串行 EEPROM，用来保存配置信息。89HPES24NT24G2 上电后可以从 EEPROM 中读取配置信息并设置 switch 的工作模式。89HPES24NT24G2 的 Slave SMBus 接口需要从插针引出。外部 I2C 主机可以通过 Slave SMBus 接口向 89HPES24NT24G2 发送命令，操作它对 EEPROM 进行擦写。

89HPES24NT24G2 有很多上电配置管脚，在进行硬件设计时应考虑他们对芯片工作模式的影响。

89HPES24NT24G2 有一组时钟配置管脚，其定义如表 4.1 所示：

表4.29 89HPES24NT24G2 的时钟配置管脚

管脚	功能
CLKMODE0	0: 端口 0 Slot 时钟频率与连接对象不同 1: 端口 0 Slot 时钟频率与连接对象相同
CLKMODE1	0: 端口 1~23 Slot 时钟频率与连接对象不同 1: 端口 1~23 Slot 时钟频率与连接对象相同
GCLKFSEL	0: 全局时钟频率 100MHz 1: 全局时钟频率 125MHz

将 CLKMODE[1:0]置为 0x03，端口 0~23 的 Slot 时钟全设置为与连接对象相同。GCLKFSEL 置为 0，选择 100MHz 时钟。

89HPES24NT24G2 有一组 switch 模式配置管脚，其定义如表 4.2 所示：

表4.30 89HPES24NT24G2 的 switch 模式配置管脚

SWMODE[3:0]	Switch 模式
0x0	单分区
0x1	带串行配置 EEPROM 的单分区
0x2	带串行配置 EEPROM 与 Jump0 初始化的单分区
0x3	带串行配置 EEPROM 与 Jump1 初始化的单分区
0x8	Reduced latency 单分区
0x9	带串行配置 EEPROM 的 reduced latency 单分区
0xA	带未连接端口的多分区
0xB	带未连接端口和 I2C 复位的多分区
0xC	带未连接端口和串行配置 EEPROM 的多分区
0xD	带未连接端口和 I2C 复位以及串行配置 EEPROM 的多分区
0xE	带未使能端口的多分区
0xF	带未连接端口和串行配置 EEPROM 的多分区

将 SWMODE[3:0]设为 0x1，即带串行配置 EEPROM 的单分区，这样就允许通过 EEPROM 对 switch 的工作模式进行重新配置。

PCIe 栈 (stack) 的也可以通过 STK2CFG 和 STK3CFG 进行配置，但栈的工作模式将通过 EEPROM 进行配置，所以可以暂不考虑。

89HPES24NT24G2 还有 3 个引脚 G1、G2 和 G3 用于选择器件类型。这里使用的是支持 PCIe Gen2 规范的 switch，所以 G2 置 1，其他置 0。

Slave SMBus 地址引脚 SSMBADDR[1:2]置为 0x3。

4.3 PCIe 通信方案软件设计

4.3.1 PCIe switch 配置

(1) SMBus 配置块格式

89HPES24NT24G2 的配置是通过它的 SMBus 接口完成的。89HPES24NT24G2 有 2 个 SMBus 接口。其中 Slave SMBus 接口可提供对软件可见的所有寄存器的访问，外部 SMBus 主机可以对它们进行读写操作。Slave SMBus 还可以用来对保存初始化信息的 EEPROM 进行编程。89HPES24NT24G2 的 Master SMBus 接口可以连接这个 EEPROM。SMBus 接口使用的通信协议与 I2C 接口相同，工作频率为 400KHz。要使 EEPROM 中的初始化信息正常被加载，SWMODE 必须配置为任意一种带串行配置 EEPROM 的模式。EEPROM 的 I2C 地址是 0b1010000X。在通过 EEPROM 进行初始化时，Master SMBus 从 EEPROM 中读取配置块（configuration blocks），并更新对应的寄存器。89HPES24NT24G2 中每个软件可见的寄存器都有唯一的全局地址，在配置块中，这些地址要右移两位，因为配置块中使用的是双字地址，而不是字节地址。Master SMBus 从 EEPROM 的 0 地址开始逐字节读取。这些字节以配置块为单位隔开，主机连续读取配置块，直到读取到配置完成块（configuration done block）的末尾。在 EEPROM 中主要用到两种配置块：单个双字初始化块和配置完成块。

表4.31 单个双字初始化块的字节序列

	bit7	bit 6	bit 5	bit 4	bit3	bit 2	bit 1	bit 0
Byte0	CFG_TYPE 0x0			0x0				
Byte1	SYSADDR[9:2]							
Byte2	SYSADDR[17:0]							
Byte3	DATA[7:0]							
Byte4	DATA[15:8]							
Byte5	DATA[23:16]							
Byte6	DATA[31:24]							

单个双字初始化块的格式如表 4.3 所示。其中 Byte0 指示配置块类型为单一双字，Byte1~2 为要设置的寄存器地址，Byte3~6 为要设置的寄存器值。

表4.32 配置完成块字节序列

	bit7	bit 6	bit 5	bit 4	bit3	bit 2	bit 1	bit 0
Byte0	CFG_TYPE 0x7			0x0				
Byte1	CHECKSUM[7:0]							

配置完成块的格式如表 4.4 所示。其中 Byte0 指示配置块类型为配置完成，Byte1 中的 checksum（校验和）能够对 EEPROM 中的内容进行校验。EEPROM 中的所有字节（包括校验和）相加，所得的 8 位计算结果为 0xFF 则通过校验。

EEPROM 可以通过 Slave SMBus 编程。Slave SMBus 的地址是 0b1110111X。向 EEPROM 写入一个字节，需要在 Slave SMBus 上发送如表 4.5 的序列：

表4.33 EEPROM 写入事务字节序列

序列编号	名称	描述
0	CCODE	命令码
1	BYTECNT	字节计数，该字节之后的字节数
2	CMD	命令
3	EEADDR	Master SMBus 上的 EEPROM 地址
4	ADDR_L	要写入 EEPROM 的地址低 8 位
5	ADDR_U	要写入 EEPROM 的地址高 8 位
6	DATA	要写入 EEPROM 的值

CCODE 的定义如表 4.6：

表4.34 CCODE 字节定义

位	名称	描述
0	END	事务结束标志
1	START	事务起始标志
4-2	FUNCTION	SMBus 操作类型 0：CSR 寄存器读写操作 1：EEPROM 读写操作

6-5	SIZE	SMBus 事务数据大小 0: Byte 1: Word 2: Block
7	PEC	错误校验使能 0: 不使能错误校验 1: 使能错误校验

CMD 的定义如表 4.7:

表4.35 CMD 字节定义

位	名称	描述
0	OP	EEPROM 操作 0: 写 EEPROM 1: 读 EEPROM
1	USA	使用特定地址 1: 使用 EEADDR 字节中地址 0: 使用 MSMBADDR 寄存器中的地址
2	-	-
3	NAERR	NACK 错误标志
5	MSS	Start&Stop 错误标志

写 EEPROM 操作的 CCODE 设置为 0x47，CMD 设置为 0x02。EEPROM 写事务的时序如图 4.2:

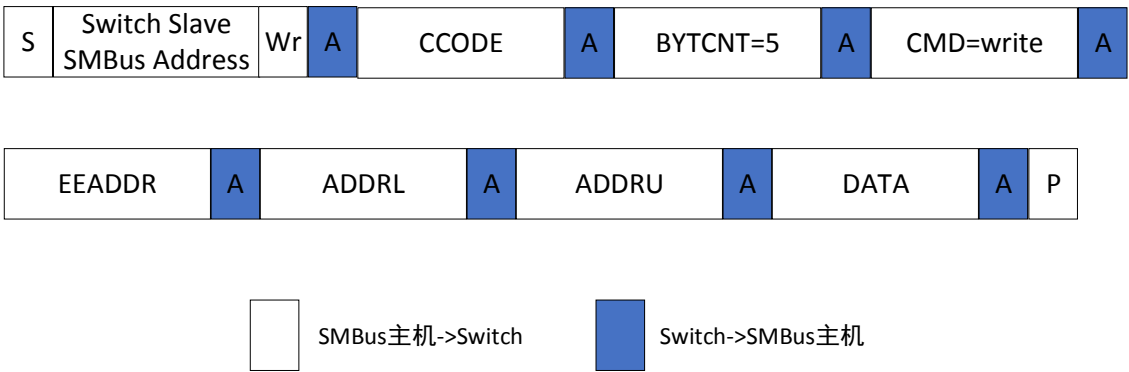


图4.23 EEPROM 写事务时序图

在 EEPROM 写事务中，数据首先发起 I2C 写请求，switch 应答，之后主机按顺序发送 EEPROM 写事务序列，switch 对每个字节进行应答，直至事务完成。

SMBus 主机的写 EEPROM 操作实现如下：

```
static void EEPROM_write(U16 addr, U8 data)
{
    I2C_start();
    I2C_writechar(0xee);
    I2C_writechar(0x47);
    I2C_writechar(0x05);
    I2C_writechar(0x02);
    I2C_writechar(0xa0);
    I2C_writechar((U8)(addr&0xff));
    I2C_writechar((U8)((addr>>8)&0xff));
    I2C_writechar(data);
    I2C_end();
    mdelay(7);
}
```

(2) 使用 SMBus 配置块进行寄存器配置

需要通过 EEPROM 进行配置的寄存器有：

Switch 控制寄存器（SWCTL），配置为 0x0011000C，解锁寄存器。Switch 控制寄存器的字段定义如表 4.8：

表4.36 Switch 控制寄存器需要配置的字段

位	名称	描述
3	REGUNLOCK	寄存器解锁 0：寄存器只读 1：寄存器可以被修改

Stack2 配置寄存器（STK2CFG），配置为 0x0000001B，端口 8~端口 15 全部工作在 x1 链路宽度。

分区 0 控制寄存器（SWPART0CTL），配置为 0x00000001，激活分区 0。分区 0 控制寄存器的字段定义如表 4.9：

表4.37 分区 0 控制寄存器需要配置的字段

位	名称	描述
1-0	STATE	分区状态控制 0x0: 不使能 0x1: 激活 0x3: 复位

端口 0 控制寄存器 (SWPORT0CTL)，配置为 0x00010002，将端口 0 设置为上游端口。端口 6~15 设置为下游端口，并依次设置设备号。端口控制寄存器字段定义如表 4.10:

表4.38 端口控制寄存器需要配置的字段

位	名称	描述
1-0	STATE	端口模式 0x1: 下游 Switch 端口 0x2: 上游 Switch 端口
14-10	DEVNUM	设备号，当端口被配置成下游端口时，该字段指定端口的设备号。

SMBus 主机的写一个配置块的操作实现如下（以配置 SWPORT0CTL 为例）:

```
EEPROM_write(addr++, 0x00);
EEPROM_write(addr++, 0x80);
checksum += 0x80;
EEPROM_write(addr++, 0xf8);
checksum += 0xf8;
EEPROM_write(addr++, 0x02);
checksum += 0x02;
EEPROM_write(addr++, 0x00);
EEPROM_write(addr++, 0x01);
checksum += 0x01;
EEPROM_write(addr++, 0x00);
```

其中变量 checksum 为校验和，用于计算配置完成块中校验和的最终值，主机的写一个配置完成块的操作实现如下:

```
EEPROM_write(addr++, 0xe0);
```

```
checksum += 0xe0;
```

```
EEPROM_write(addr++, ~checksum);
```

寄存器配置完成后，switch 的分区 0 中将包括一个上游端口和 10 个下游端口。工作模式如图 4.3 所示。

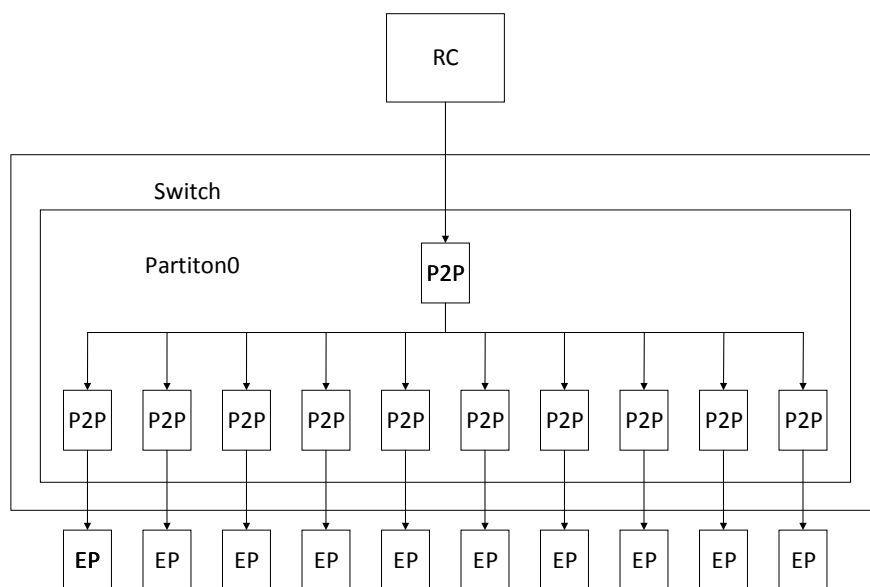


图4.24 89HPES24NT24G2 的工作模式

4.3.2 MPC8377 PCIe 控制器配置

在多个 PowerPC 处理器的 PCIe 通信系统中，作为 RC 的 MPC8377 PCIe 控制器配置与第三章中 RC 处理器的配置基本相同。其它的 MPC8377 作为 EP 设备，它们的 PCIe 控制器配置与 RC 有所区别。

(1) 串行串解器和工作模式配置

首先在配置串行串解器工作模式时，RC 端处理器只需要一路 PCIe x1，EP 端处理器需要一路 PCIe x1 和一路 SATA。

作为 EP 的 MPC8377 的 PCIe 控制器应工作在 EP 设备类型下，要在 PECR 中将 DEV_TYPE 设为 EP，并对链路、CSB 桥和 CSR 进行软复位。

(2) 本地访问窗口和地址译码窗口配置

EP 的本地访问窗口和 outbound 窗口的设置与 RC 相同，但 inbound 窗口的设置与 RC 有所区别。MPC8377 的 CSB 桥有 4 个 EP inbound 窗口，其中窗口 0 对应 BAR0 空间，窗口 1 对应 BAR1 空间，窗口 2 对应 BAR2 和 BAR3 空间，窗口 3 对应 BAR4

和 BAR5 空间。当 PCIe inbound 事务命中有效的地址窗口时，该事务会通过 CSB 桥被传递至 CSB 总线。每个 inbound 窗口有 1 个窗口译码地址寄存器 PEX_EPIWTAR。Inbound 窗口译码地址寄存器的字段定义如表 4.11：

表4.39 Inbound 窗口译码地址寄存器需要配置的字段

位	名称	描述
31-12	TA	译码地址，被映射至 PCIe inbound 窗口的 CSB 基地址的 31-12 位。低 12 为固定为 0。
0	EN	使能该窗口

RC 只访问 EP 设备的 BAR0 空间，所以在 uboot 中，将 EPIWTAR0 设置为 0x0C800001，EPIWTAR1~3 设置为 0x00000000（不使能）。这样在 Linux 系统中将 0x0C800000 映射成虚拟地址，就可以通过该虚拟地址访问 BAR0 空间。

设置好译码地址后，还要设置 BAR 空间的属性。BAR 配置寄存器组包括 BAR 空间大小寄存器 PEX_BAR_SIZEL、BAR 选择寄存器 PEX_BAR_SEL、BAR 预取配置寄存器 PEX_BAR_PF。BAR 空间大小寄存器、BAR 选择寄存器和 BAR 预取配置寄存器的字段定义如表 4.12、4.13、4.14：

表4.40 BAR 空间大小寄存器需要配置的字段

位	名称	描述
31-12	MASK	BAR 掩码，掩码覆盖 MASK[31:m]时，BAR 空间大小为 2^m 字节，最大 4G 字节。

表4.41 BAR 选择寄存器需要配置的字段

位	名称	描述
1-0	SEL	选择要编程的 PEX_BAR_SIZEL。 00: PEX_BAR_SIZEL 指向 BAR0 01: PEX_BAR_SIZEL 指向 BAR0 10: PEX_BAR_SIZEL 指向 BAR0 11: PEX_BAR_SIZEL 指向 BAR0

表4.42 BAR 预取寄存器需要配置的字段

位	名称	描述
3-0	PF	可预取

		PF[3]: BAR3 可预取 PF[2]: BAR2 可预取 PF[1]: BAR1 可预取 PF[0]: BAR0 可预取
--	--	--

设置 BAR0 空间的大小时，先将 PEX_BAR_SEL 设置为 0x00000000，使 PEX_BAR_SIZE_L 指向，随后设置 PEX_BAR_SIZE_L 为 0xffc00000，即 BAR0 空间的大小为 4MB。PEX_BAR_PF 设置为 0x00000001，即 BAR0 可预取。

(3) type0 配置头设置

设置 PEX_SSVID_UPDATE 寄存器，将 type0 配置头中的 Subsystem ID 更改为 0xABCD。更改这些 ID 的目的是防止驱动程序将 RC 和 EP 混淆，RC 和 EP 的 Vendor ID 和 Device ID 相同，若不用 Subsystem ID 和加以区别，驱动程序会认为 RC 和 EP 是相同的设备。将 Command 的如下位置为 1：0（使能 I/O 空间响应）、1（使能内存空间响应）、2（使能总线主机功能）、6（奇偶校验）、8（使能 SERR）。将 Status 清零（写 1 清零），并将 Latency Timer 设为 0x80，Cache Line Size 设为 0x08。Vendor ID 默认为 0x1957（Freescall），Device ID 默认为 0x00C6（MPC8377E），Vendor ID 和 Device ID 不可修改。Type0 配置头描述如表 4.15。

表4.43 PCIe type0 配置头描述

名称				地址偏移
Device ID		Vendor ID		00
Status		Command		04
Class Code			Revision ID	08
BIST	Header Type	Latency Timer	Cache Line Size	0C
Base Address Register				10
				14
				18
				1C
				20
				24
				28
Subsystem ID		Subsystem Vendor ID		2C

				30
			Capabilities Point	34
Expansion ROM Base Address				38
MAX_LAT	MIN_GNT	Interrupt Pin	Interrupt Line	3C

完成了 type0 配置头的设置后，就可以检查 LTSSM 状态机的状态了。读取 PEX_LTSSM_STAT 寄存器可以获取 LTSSM 状态机的状态码。若状态码为 0x16(L0)，则表示 PCIe 链路已训练完成，进入正常工作模式。至此 uboot 中的 PCIe 初始化操作完毕。

4.3.3 EP 设备的驱动程序设计

EP 设备驱动程序与第三章所讨论的驱动程序的框架是一致的，但二者的具体实现不尽相同。

(1) 驱动程序中的结构体定义

首先驱动程序所对应的设备是作为 EP 的 MPC8377:

```
static struct pci_device_id pciecard_pci_tbl [] __initdata = {
    {PCIeCARD_VENDOR_ID, PCIeCARD_DEVICE_ID,\
    PCI_ANY_ID, 0xabcd, 0, 0, 8},
    {0,}
};
```

PCIeCARD_VENDOR_ID 定义厂商 ID 为 0x1957 (Freescale) , PCIeCARD_DEVICE_ID 定义设备 ID 为 0x00C6 (MPC8377E)。Subsystem ID 为 0xABCD，即作为 EP 的 MPC8377。

RC 最多连接 10 个 EP，可以定义一个 EP 设备结构体，并用长度为 10 的数组管理所有 EP 设备结构体：

```
struct pciecard_dev {
    u8 *virt_base_bar0;
    u32 hdwr_base_bar0;
    u32 hdwr_end_bar0;
    u32 len_bar0;
    dev_t pciecard_dev_no;
    struct cdev cdev;
    struct class *dev_class;
```



```
};
static struct pciecard_dev pciecard_dev[10];
```

(2) 探测函数设计

在探测函数中，除完成 PCIe 设备初始化和字符设备初始化外，还增加了在探测设备时创建设备文件节点的功能。此外，若连接了 10 个 EP 设备，则探测函数要调用 10 次，并对 10 个设备分配不同的内存，创建不同的设备节点。探测函数实现如下：

```
static int __devinit pciecard_probe(struct pci_dev *pdev, const struct pci_device_id
*pci_id)
```

```
{
```

激活设备：

```
    if(pci_enable_device(pdev)){
        printk(KERN_WARNING "pciecard not enabled\n");
        return -EFAULT;
    }
```

使能主机功能，设置 DMA 掩码：

```
    pci_set_master(pdev);
    pci_set_dma_mask(pdev, (u64) 0xffffffffffffff);
```

为设备申请内存空间：

```
    if(pci_request_regions(pdev, MODNAME)){
        printk(KERN_WARNING "pci_request_regions failed\n");
        return -EFAULT;
    }
```

映射 BAR0 空间：

```
    pciecard_dev[card_num].hdwr_base_bar0 = pci_resource_start(pdev, 0);
    pciecard_dev[card_num].hdwr_end_bar0 = pci_resource_end(pdev, 0);
    pciecard_dev[card_num].len_bar0 = pci_resource_len(pdev, 0);
    pciecard_dev[card_num].virt_base_bar0 = \
ioremap(pciecard_dev[card_num].hdwr_base_bar0, \
pciecard_dev[card_num].len_bar0);
```

初始化字符设备：

```
    cdev_init(&pciecard_dev[card_num].cdev, &pciecard_fops);
    alloc_chrdev_region(&pciecard_dev[card_num].pciecard_dev_no, \
```

```
0, 1, MODNAME);
```

创建设备类和设备文件节点：

```
pciecard_dev[card_num].dev_class =\
class_create(THIS_MODULE, dev_names[card_num]);
device_create(pciecard_dev[card_num].dev_class, NULL, \
pciecard_dev[card_num].pciecard_dev_no, NULL, dev_names[card_num]);
pciecard_dev[card_num].cdev.owner = THIS_MODULE;
```

将字符设备加入设备列表：

```
if(cdev_add(&pciecard_dev[card_num].cdev, \
pciecard_dev[card_num].pciecard_dev_no, 1)){
    printk(KERN_WARNING "cdev_add error\n");
    return -EFAULT;
}
```

下次调用探测函数时，card_num 已经增加 1，避免添加重复的设备：

```
card_num++;
return 0;
}
```

(3) 文件操作

在设备文件的打开操作中，要将设备文件所对应的 pciecard_dev 结构体作为文件句柄的私有数据，这样就可以通过该结构体找到相应设备的内存地址。设备文件打开函数的实现如下：

```
static int pciecard_open(struct inode *inode, struct file *filp)
{
    struct pciecard_dev *dev;
    dev = container_of(inode->i_cdev, struct pciecard_dev, cdev);
    filp->private_data = dev;
    return 0;
}
```

设备文件写操作的主要功能是将用户空间的数据写入 EP 的 BAR0 空间。设备文件写函数的实现如下：

```
static int pciecard_write(struct file *filp, const char __user *buf, size_t count, loff_t
*offset)
{

```

```

struct pciecard_dev *dev = filp->private_data;
unsigned long p = *offset;
if(p >= WR_SIZE){
    printk(KERN_WARNING "invalid write position\n");
    return 1;
}
if((count <= (WR_SIZE - p)) && (count != 0)){
    copy_from_user((dev->virt_base_bar0 + p), buf, count);
}
else{
    printk(KERN_WARNING "invalid write count\n");
    return 1;
}
return 0;
}

```

设备文件读操作的主要功能是将 EP 设备 BAR0 空间中的数据读取到用户空间。设备文件读函数的实现如下：

```

static int pciecard_read(struct file *filp, char __user *buf, size_t count, loff_t *offset)
{
    struct pciecard_dev *dev = filp->private_data;
    unsigned long p = *offset;
    if(p >= WR_SIZE){
        printk(KERN_WARNING "invalid read position\n");
        return 1;
    }
    if((count <= (WR_SIZE - p)) && (count != 0)){
        copy_to_user(buf, (dev->virt_base_bar0 + p), count);
    }
    else{
        printk(KERN_WARNING "invalid read count\n");
        return 1;
    }
    return 0;
}

```

RC 与 EP 通信时未使用中断方式，所以不需要绑定中断处理函数。

卸载设备时，除了释放资源，删除字符设备外，还要删除设备文件节点：

```
static int __devexit pciecard_remove(struct pci_dev *pdev)
{
    card_num--;
    unregister_chrdev_region(pciecard_dev[card_num].pciecard_dev_no, 1);
    device_destroy(pciecard_dev[card_num].dev_class, \
pciecard_dev[card_num].pciecard_dev_no);
    class_destroy(pciecard_dev[card_num].dev_class);
    iounmap(pciecard_dev[card_num].virt_base_bar0);
    pci_release_regions(pdev);
    pciecard_dev[card_num].virt_base_bar0 = NULL;
    pci_disable_device(pdev);
    cdev_del(&pciecard_dev[card_num].cdev);
    return 0;
}
```

将驱动程序源代码编译成内核模块，以便在 Linux 系统运行时装载驱动。

4.3.4 RC 端应用程序设计

运行在 RC 端应用程序是基于运行在 MPC8377 上的嵌入式 Linux 操作系统的。主要完成参数与缓冲区初始化，以及对数据文件和设备的操作。

EP 和 RC 之间的 PCIe switch 在软件层面是透明的，RC 可直接对 EP 的 BAR 空间进行访问。10 个 EP 设备对应的设备名是 pciecard n (n 为 0 到 9)。

RC 与 EP 的数据传输没有采用 DMA 的方式，所有数据交换都在 EP 的 BAR0 空间中完成。应用程序首先打开 EP 设备文件。在向 RC 发送数据时，应对相应的设备文件进行写操作；从 RC 读取数据时，应对相应的设备文件进行读操作。

由于没有采用 DMA 的传输方式，程序不能通过 DMA 完成中断得知读写操作已经完成。为解决这一问题，在程序设计时，约定 EP 设备 BAR0 空间的第一个字节为读写状态标识。若读写状态标识为 0x00，表示没有进行 BAR0 空间的任何读写操作，此时可以对 BAR0 空间发起读或写；若读写状态标识 0xff，表示对 BAR0 空间的写操作完成，可以将 BAR0 空间中的数据读走，读走数据后，读取数据的一方需要将读写状态标识重置为 0x00，表一次读写操作已经完成，可以发起下一次读写操作；若读写状态标识 0xaa，表示一个完整的文件已传输完成，不需要继续发起读写。

在 PCIe 传输功能测试程序中，实现了将 RC 的文件系统中的本地文件发送至 EP

的操作:

```
int fd_dev;
int fd_src;
unsigned int pos = 0;
char c = 0;
unsigned int w_reg, r_reg;
char *buf_addr = NULL;
struct stat file_stat;
int i = 0;
buf_addr = (char*)malloc(WRITE_SIZE);
if(buf_addr == NULL)
    printf("malloc failed\n");
```

以上代码声明文件句柄，初始化数据缓冲区。

```
fd_dev = open("/dev/pciecard0", O_RDWR);
fd_src = open("/mnt/hdd0/let", O_RDWR);
stat("/mnt/hdd0/let", &file_stat);
while(pos < file_stat.st_size){
    lseek(fd_src, pos, 0);
    pos += WRITE_SIZE - 1;
    lseek(fd_dev, 0, 0);
    read(fd_dev, buf_addr, 1);
    while(*buf_addr != 0x00){
        lseek(fd_dev, 0, 0);
        read(fd_dev, buf_addr, 1);
        printf("%x\n", *buf_addr);
    }
}
```

在以上循环中等待读写标志清零。

```
if(pos >= file_stat.st_size){
    read(fd_src, buf_addr + 1, WRITE_SIZE - 1 - (pos - file_stat.st_size));
    *buf_addr = 0xaa;
}
else{
```

```

        read(fd_src, buf_addr + 1, WRITE_SIZE - 1);
        *buf_addr = 0xff;
    }

```

每次读写操作传输 WRITE_SIZE 个字节。每次读写操作后，判断文件是否传输完成，若文件传输完成则将读写标志置为 0xaa，否则将读写标志置为 0xff。

```

        lseek(fd_dev, 1, 0);
        write(fd_dev, buf_addr + 1, WRITE_SIZE - 1);
        lseek(fd_dev, 0, 0);
        write(fd_dev, buf_addr, 1);
    }

```

文件传输完成后退出读写操作循环。

```

    close(fd_dev);
    close(fd_src);
    free(buf_addr);
    return 0;

```

关闭文件，释放缓冲区内存。

4.3.5 EP 端应用程序设计

EP 端应用程序的主要工作是 PCIe 通信以及数据文件的创建和读写。

EP 的 PCIe BAR0 inbound 窗口是映射在物理地址 0x0C800000 的，EP 端应用程序要访问该窗口，需要调用 Linux 系统函数 mmap()，将从 0x0C800000 开始的 4MB 空间映射至虚拟内存空间，应用程序通过这段虚拟内存就可以访问 BAR0 空间了。

EP 端的 PCIe 通信操作与 RC 端类似，只是在 RC 端程序中对设备文件的读写，在 EP 端程序中变成了对特定内存区域的读写，具体过程这里不再赘述。

4.4 PCIe 通信功能测试

本节主要对通信系统的 PCIe 传输和文件存储功能进行测试。

测试前连接好调试串口和电源，并上电启动。在 RC 处理器打印出的 uboot 启动信息如图 4.4。可以看到，PCIe 控制器 2 (PCIe1) 的 LTSSM 状态机已经处于 16 (L0) 状态，说明 PCIe 链路已经正常建立。配置头类型为 type 1，设备类型为 RC 设备。

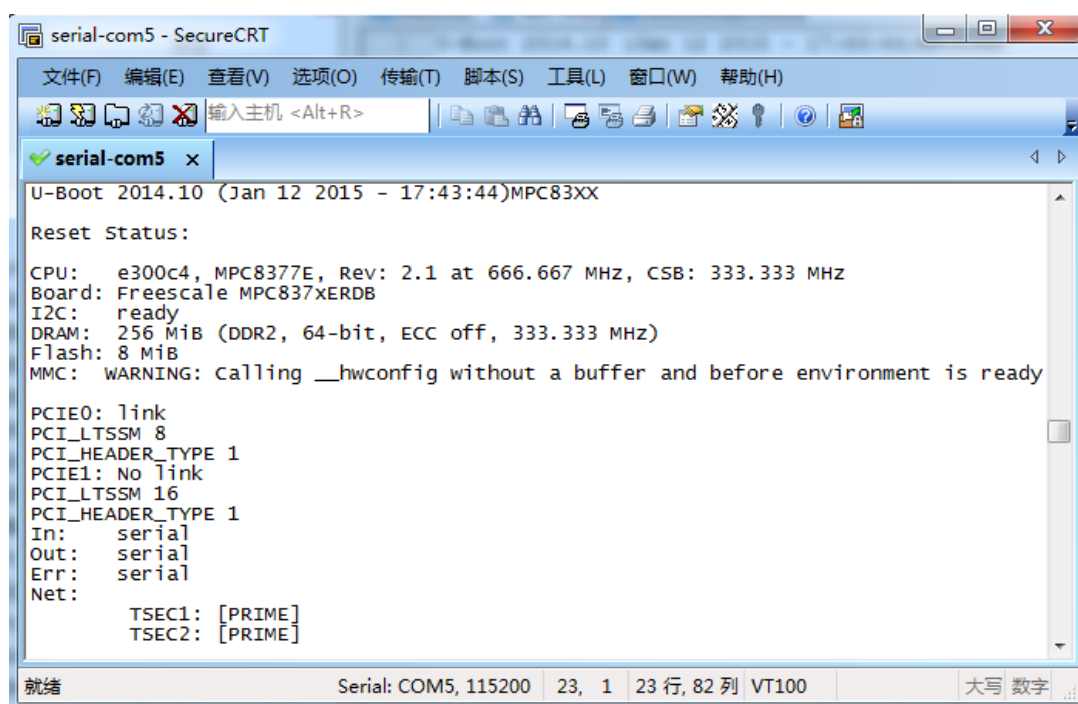


图4.25 RC 端启动信息

在 EP 处理器打印出的 uboot 启动信息如图 4.5。可以看到, PCIe 控制器 2 (PCIe1) 的 LTSSM 状态机已经处于 16 (L0) 状态, 说明 PCIe 链路已经正常建立。配置头类型为 type 0, 设备类型为 EP 设备。

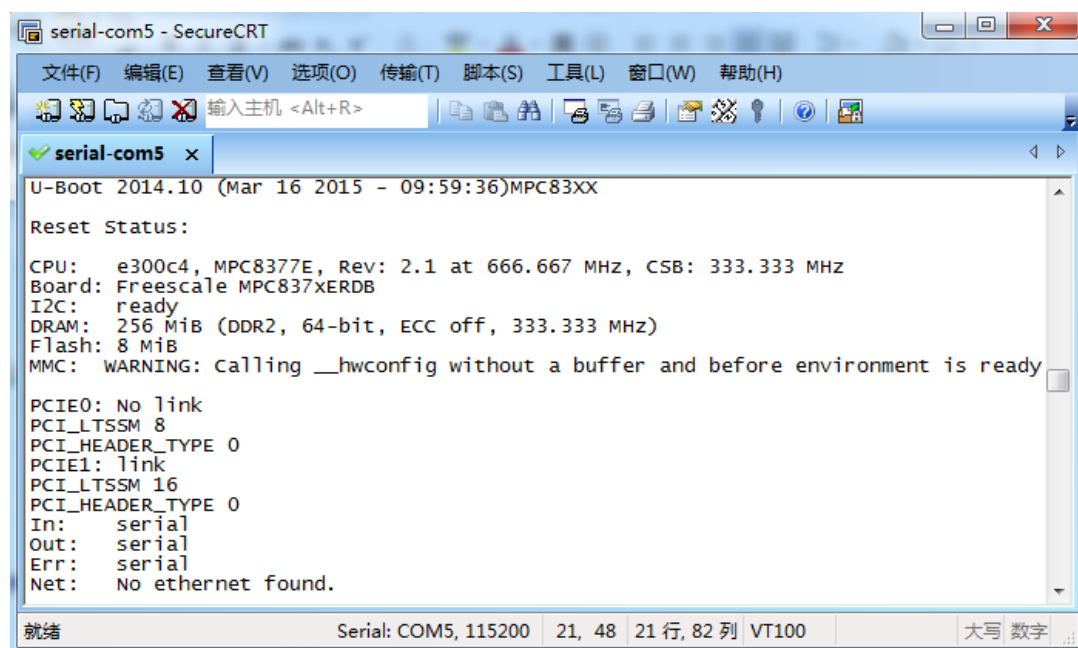
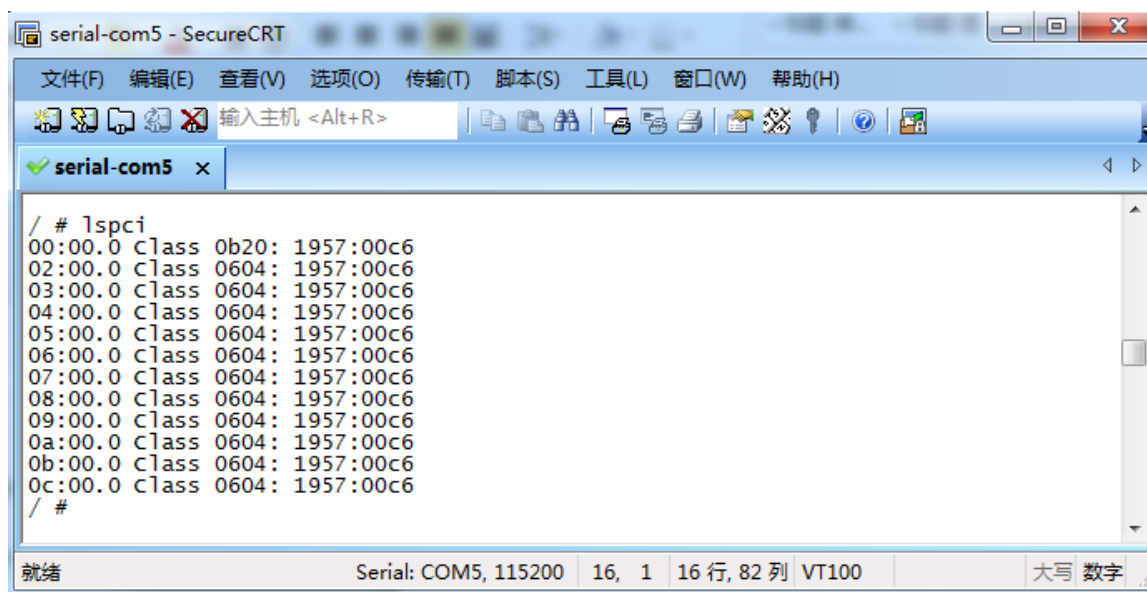


图4.26 EP 端启动信息

在 RC 端的 Shell 中输入 lspci 命令, 可以看到已经与 RC 建立连接 10 个 MPC8377

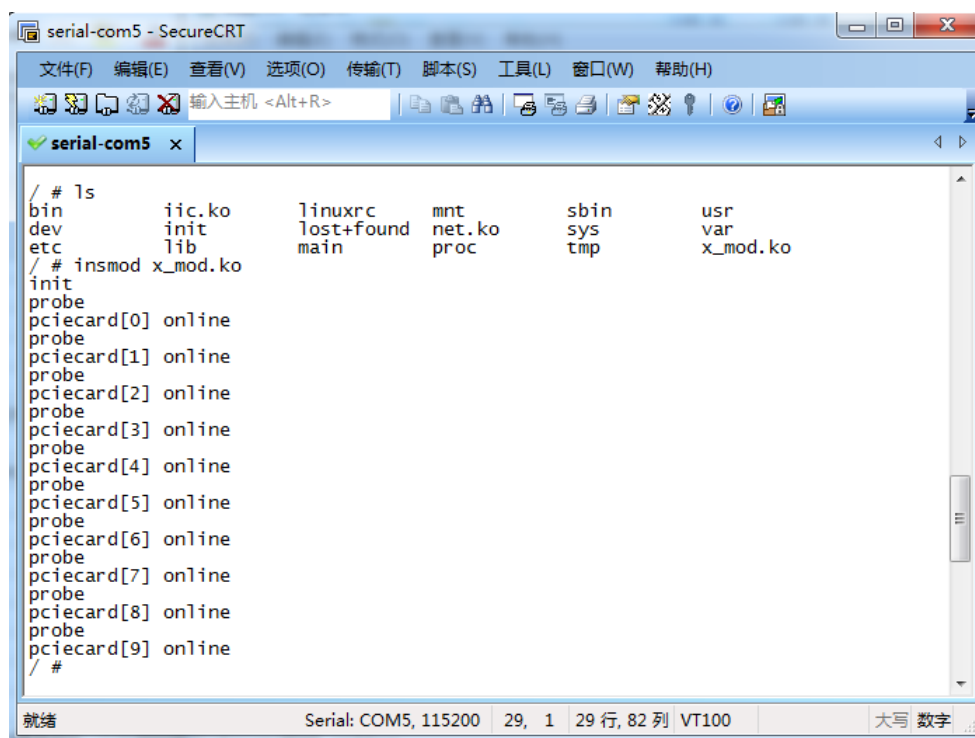
EP 设备（列表中前两个设备为 RC），设备信息如图 4.6 所示。



```
/ # lspci
00:00.0 class 0b20: 1957:00c6
02:00.0 class 0604: 1957:00c6
03:00.0 class 0604: 1957:00c6
04:00.0 class 0604: 1957:00c6
05:00.0 class 0604: 1957:00c6
06:00.0 class 0604: 1957:00c6
07:00.0 class 0604: 1957:00c6
08:00.0 class 0604: 1957:00c6
09:00.0 class 0604: 1957:00c6
0a:00.0 class 0604: 1957:00c6
0b:00.0 class 0604: 1957:00c6
0c:00.0 class 0604: 1957:00c6
/ #
```

图4.27 PCI 设备列表

将编译好的驱动程序模块复制到 RC 端的文件系统中，用 `insmod` 命令加载。图 4.7 中的信息表明 10 个 EP 设备的驱动程序全部加载成功。



```
/ # ls
bin      iic.ko      linuxrc     mnt         sbin        usr
dev      init        lost+found  net.ko      sys         var
etc      lib         main        proc        tmp         x_mod.ko
/ # insmod x_mod.ko
init
probe
pciecard[0] online
probe
pciecard[1] online
probe
pciecard[2] online
probe
pciecard[3] online
probe
pciecard[4] online
probe
pciecard[5] online
probe
pciecard[6] online
probe
pciecard[7] online
probe
pciecard[8] online
probe
pciecard[9] online
/ #
```

图4.28 装载设备驱动

驱动程序识别到 EP 设备后，会自动为每个识别到的设备创建相应的设备文件节点。驱动加载完毕后，即可在文件系统的/dev 目录下看到 pciecard0~9 设备文件节点，如图 4.8。

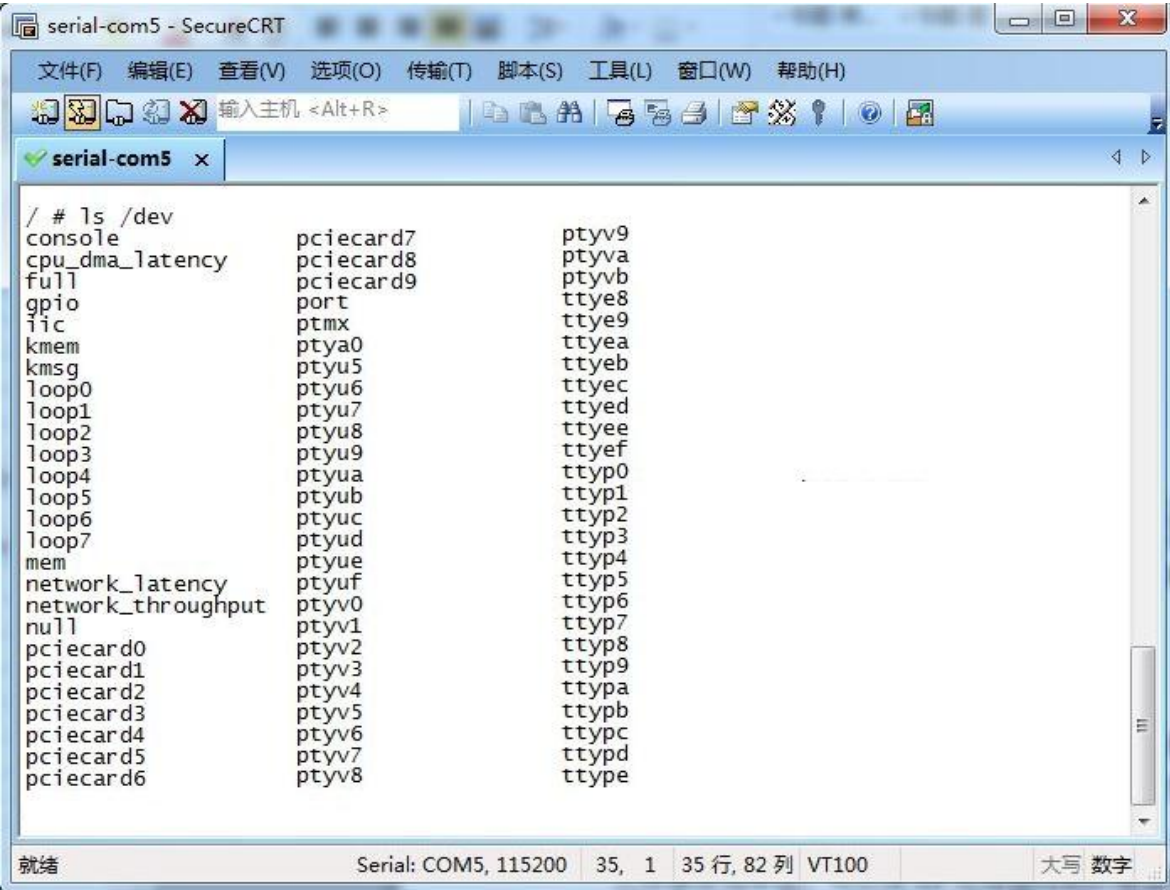
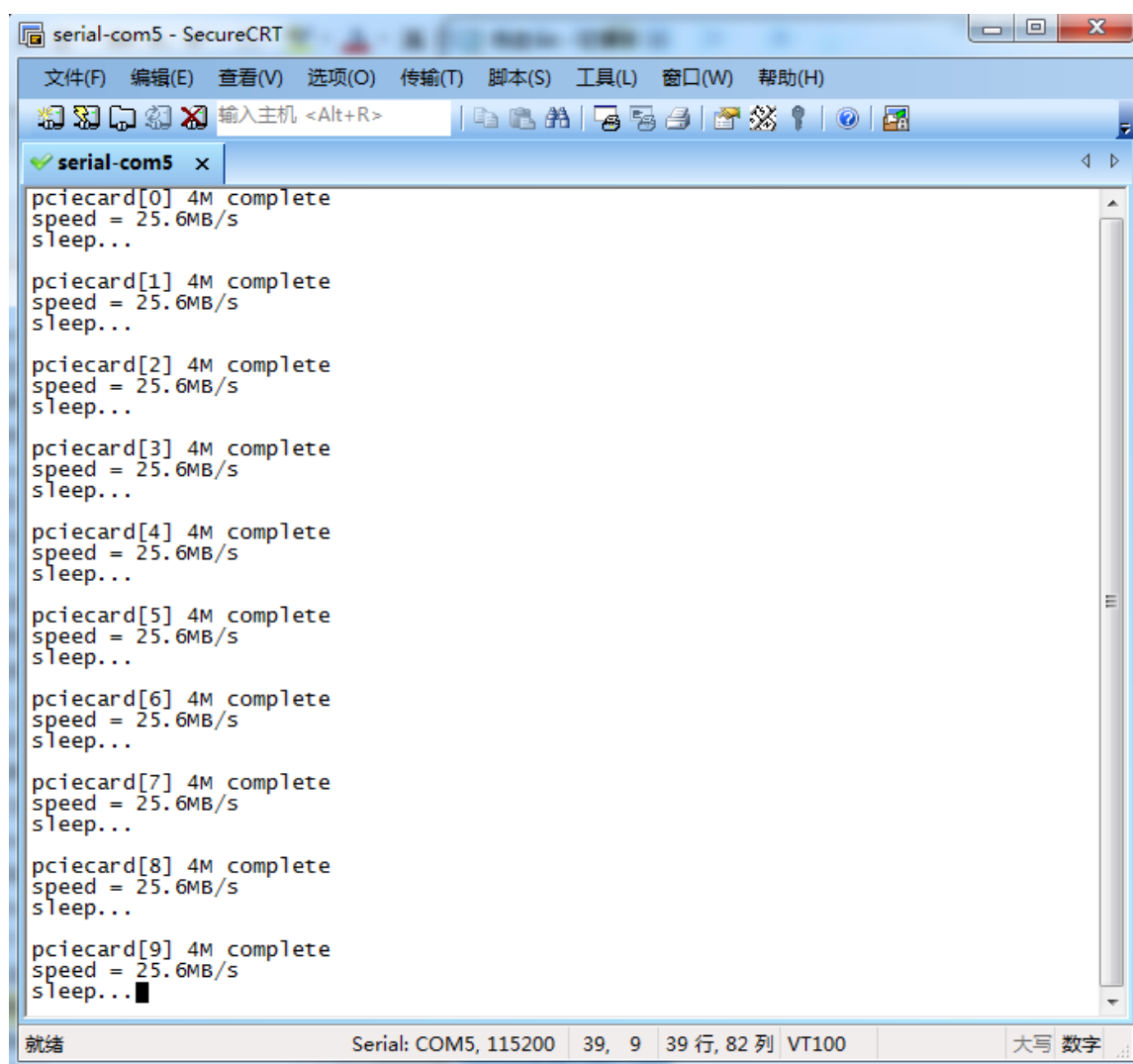


图4.29 设备文件列表

在 RC 处理器和 EP 处理器上运行各自的测试应用程序。RC 端程序完成初始化工作后会进入主循环，将本地文件系统中的一段视频文件同时发送给 10 个 EP 设备。EP 端测试程序则接收视频文件数据，并保存到硬盘。实际在微观上，RC 对每个 EP 的操作是分时轮流进行的。

文件保存完毕后，可以将 EP 处理器挂载的硬盘取下，安装在计算机上，打开其中的视频文件检视。

经实际测试，数据的平均传输速度在 25MB/s 左右，没有达到 PCIe 1.0 x1 的正常带宽，这主要是受到硬盘写入速度的限制和写入操作间隔的影响。程序运行时打印的调试信息如图 4.9 所示。



```
serial-com5 - SecureCRT
文件(F) 编辑(E) 查看(V) 选项(O) 传输(T) 脚本(S) 工具(L) 窗口(W) 帮助(H)
输入主机 <Alt+R>
serial-com5 x
pciecard[0] 4M complete
speed = 25.6MB/s
sleep...

pciecard[1] 4M complete
speed = 25.6MB/s
sleep...

pciecard[2] 4M complete
speed = 25.6MB/s
sleep...

pciecard[3] 4M complete
speed = 25.6MB/s
sleep...

pciecard[4] 4M complete
speed = 25.6MB/s
sleep...

pciecard[5] 4M complete
speed = 25.6MB/s
sleep...

pciecard[6] 4M complete
speed = 25.6MB/s
sleep...

pciecard[7] 4M complete
speed = 25.6MB/s
sleep...

pciecard[8] 4M complete
speed = 25.6MB/s
sleep...

pciecard[9] 4M complete
speed = 25.6MB/s
sleep...
就绪 Serial: COM5, 115200 39, 9 39 行, 82 列 VT100 大写 数字
```

图4.30 PCIe 通信应用程序运行情况

第五章 总结与展望

本文从嵌入式系统对 PCIe 总线的性能需求入手,并利用 FPGA 中的 PCIe 接口逻辑核实现 PCIe EP 设备,并以 MPC8377 PowerPC 处理器作为 PCIe RC,接收 FPGA 产生的数据。FPGA 与嵌入式 PowerPC 处理器的 PCIe 通信系统方案的可行性,为多个嵌入式 PowerPC 处理器的 PCIe 通信系统的设计提供了基础和经验。在多个嵌入式 PowerPC 处理器的 PCIe 通信系统中,一颗处理器作为 RC 设备,并通过 switch 与其他 EP 处理器建立 PCIe 链路,实现了一对多的 PCIe 通信。目前关于多嵌入式处理器的 PCIe 通信技术的研究还很少,本文提出的这种方案,具有一定的参考价值。

本文所完成的工作主要有:

(1) 完成了 MPC8377 处理器、XC6VLX240T FPGA 以及 89HPES24NT24G2 switch 的硬件系统设计。良好的硬件设计能够为系统稳定运行提供保障,同时也是继续后续软件开发的必要平台。

(2) 实现了 MPC8377 处理器的 PCIe RC 设备功能,并使用 FPGA 作为 PCIe EP 设备,与 RC 建立连接,验证了 MPC8377 作为 PCIe RC 的可行性,并在测试中实现了对 FPGA 产生的数据的接收。

(3) 以 PCIe 与 FPGA 的 PCIe 通信系统 RC 方案为基础,进一步对多个嵌入式 PowerPC 处理器的 PCIe 通信进行探索,充分发掘了对 MPC8377 的 PCIe EP 功能。同时从理论上升到实践,使用 PCIe switch 对 PCIe 总线进行了链路扩展,实现了一个 RC 设备与多个 EP 设备的通信。

本文实现了基于嵌入式平台的 PCIe 通信和 PCIe 链路扩展,但这些系统仍存在许多不足之处,在如下方面仍有较大的改进空间:

(1) 在本文实现的 PCIe 通信中,PCIe 总线的带宽并没有被完全利用,主要的原因是两次 PCIe 读写操作之间的间隔时间较长,PCIe 总线常处于空闲状态。另外测试时的硬盘写入速率也造成了一定的瓶颈。PCIe 通信应用程序还有很多可优化之处。

(2) 本文实现的 PCIe 通信都是基于 PCIe 1.0 规范的 x1 宽度链路,在解决了传输速率瓶颈问题后,若需要进一步提高传输速度,可以将 MPC8377 的 PCIe 链路宽度设置为 x2,或将 MPC8377 换成支持更高 PCIe 规范版本和更大链路宽度的处理器。

(3) FPGA 与 PowerPC 通信时,PowerPC 需要控制 FPGA 的 DMA 模块,操作较为繁琐。可以借鉴多 PowerPC 的 PCIe 通信方式,改进 FPGA 上的逻辑设计,使 PowerPC 可以通过直接读写 BAR 空间的方式进行数据传输。

参考文献

- [1] 贾振国. 基于 DSP 的嵌入式网络视频服务的管理与控制技术[D]. 西安: 西安电子科技大学, 2011.
- [2] Eric Bogatin. 信号完整性分析[M]. 北京: 电子工业出版社, 2010.
- [3] 王恒亮. 基于 MPC8377E 的网络电子存储阵列关键技术研究[D]. 西安: 西安电子科技大学, 2015.
- [4] 刘佳音. 基于 PCI 总线的合成孔径雷达图像实时显示系统[D]. 北京: 中国科学院电子学研究所, 2006.
- [5] Michael Meyers. CompTIA A+ Certification All-in-One Exam Guide[M]. McGraw Hill Professional, 2012.
- [6] 刘金良. 基于现代微机面向软件类计算机原理实验设备的研究与设计[D]. 天津: 南开大学, 2005.
- [7] 贾真, 林清. PCI 总线应用设计与研究[J]. 现代电子技术, 2008(10):85-87.
- [8] 马旭. 基于 PCI Express 协议的 WDM 驱动程序开发[D]. 西安: 西安电子科技大学, 2013.
- [9] 毛祖秋. 基于 OpenSG 的集群绘制系统[D]. 杭州: 浙江大学, 2006.
- [10] 郭绍日, 张振宇. PCI Express 总线技术剖析[J]. 电子测试, 2004(11):55-60.
- [11] 熊信璇. 基于 PCI 总线的立体显示技术研究[D]. 武汉: 华中师范大学, 2011.
- [12] 周俊峰. 基于 PCI Express 总线数据采集记录系统硬件设计[D]. 成都: 电子科技大学, 2009.
- [13] 胡建超. 基于 FPGA 的 PCI 图像传输与处理卡设计及实现[D]. 西安: 西安电子科技大学, 2012.
- [14] 徐佳谋. 基于 PCI Express 总线的高性能嵌入式系统研究[D]. 北京: 北京工业大学, 2009.
- [15] 王齐. PCI Express 体系结构导读[M]. 北京: 机械工业出版社, 2010.
- [16] 魏海波. 基于 PCI-E 的阵列信号处理系统硬件研究[D]. 重庆: 重庆大学, 2014.
- [17] 李楚洲. 基于 PCIE 总线的 CCSDS 标准卫星数据采集系统的设计与实现[D]. 广州: 华南理工大学, 2011.
- [18] 主宾. 通用 CPU 在高速电路中的应用[D]. 上海: 复旦大学, 2012.
- [19] 姚铭. 高密度高速存储系统设计与实现[D]. 西安: 西安电子科技大学, 2013.
- [20] Freescale. MPC8379E PowerQUICC II Pro Integrated Host Processor Family Reference Manual[M]. USA:Freescale, 2009.
- [21] IDT. 89HPES12NT12G2 PCI Express Switch User Manual[M]. USA:IDT, 2013.
- [22] 陈世平. 高分辨率卫星遥感数据传输技术发展的若干问题[J]. 空间电子技术, 2003(3):1-5.
- [23] Texas Instruments. 1.6 to 2.7 Gbps transceiver[M]. USA:Texas Instruments, 2001.

- [24] Xilinx. Virtex-6 FPGA Configuration User Guide[M]. USA:Xilinx, 2012.
- [25] IDT. ICS841484I HCSL Clock Generator User Manual[M]. USA:IDT, 2012.
- [26] 彭涛. 基于 PCIe 总线的高速数据传输技术研究[D]. 西安: 西安电子科技大学, 2015.
- [27] Johnthan Corbet, Alessandro Rubini, Greg Kroah-Hartman. Linux Device Drivers[M]. USA: O'Reilly Media, 2005.
- [28] 宋宝华. Linux 设备驱动开发详解[M]. 北京: 人民邮电出版社, 2010.
- [29] Marvell. 88E1111 Product Brief[M]. USA:Marvell, 2009.
- [30] IDT. ICS9DB803DI Eight Output Differential Buffer for PCIe Gen 2[M]. USA:IDT, 2008.
- [31] IDT. ICS9FG104 Frequency Generator User Manual[M]. USA:IDT, 2007.

致谢

硕士研究生阶段的学习和生活已经接近尾声了，在这两年半的学习生活中，课题组的老师们和同学们给予了我很大的帮助和关怀，在他们的帮助下我在生活上和专业技能上都有了很大的进步，在这里对他们致以最诚挚的感谢。

首先要感谢我的导师，武斌副教授。武老师有丰富的工程经验，在完成实验室的项目过程当中，武老师给我们提供了很大的帮助，帮我们解决了很多技术难题，在科研思路引导我们，提高了我们的科研能力，丰富了我们的项目经验。当我们在项目中犯错时，也积极鼓励我们，帮助我们解决问题，提高了我们独立思考与独立解决问题的能力。在论文的完成期间，武老师提出了很多宝贵意见。在这里向武老师表示衷心的感谢！

此外要感谢课题组的李鹏教授和冯小平教授，李老师和冯老师是课题组中经验最丰富，资历最高的两位老师，有很好的大局观。在项目进行的初期，能够给出最合理的项目实现方案；每当在项目中遇到棘手问题时，李老师和冯老师都帮忙解决技术难题，让我们在科研工作中少走了很多弯路。并且还要感谢李老师和冯老师为我们提供了非常优越的科研环境，在实验室中提供了专业的精密仪器和设备，让我们提高了平时科研工作的效率。在此向李老师和冯老师表示衷心的感谢！

感谢实验室一同工作的张耀军、王铜、强景同学，他们在项目完成去给我提供了很多建设性意见，在我遇到技术瓶颈时给予我鼓励，让我坚持了下来并最终克服了技术问题。感谢彭涛、王恒亮师兄细心的给我讲解项目中的技术问题与技术难点，让我更快的熟悉了项目内容，在最快的时间内融入了项目组，实现项目中未完成部分以及功能的升级。在此对以上在科研中帮助过我的同学及师兄们。

最后，衷心感谢为审阅论文而付出辛勤劳动的各位专家和评委们。



西安电子科技大学
XIDIAN UNIVERSITY

地址：西安市太白南路2号

邮编：710071

网址：www.xidian.edu.cn